



COMP 4300

Intro to Game Programming

Lecture 7

Collision Detection / Resolution
Axis-Aligned Bounding Boxes (AABB)

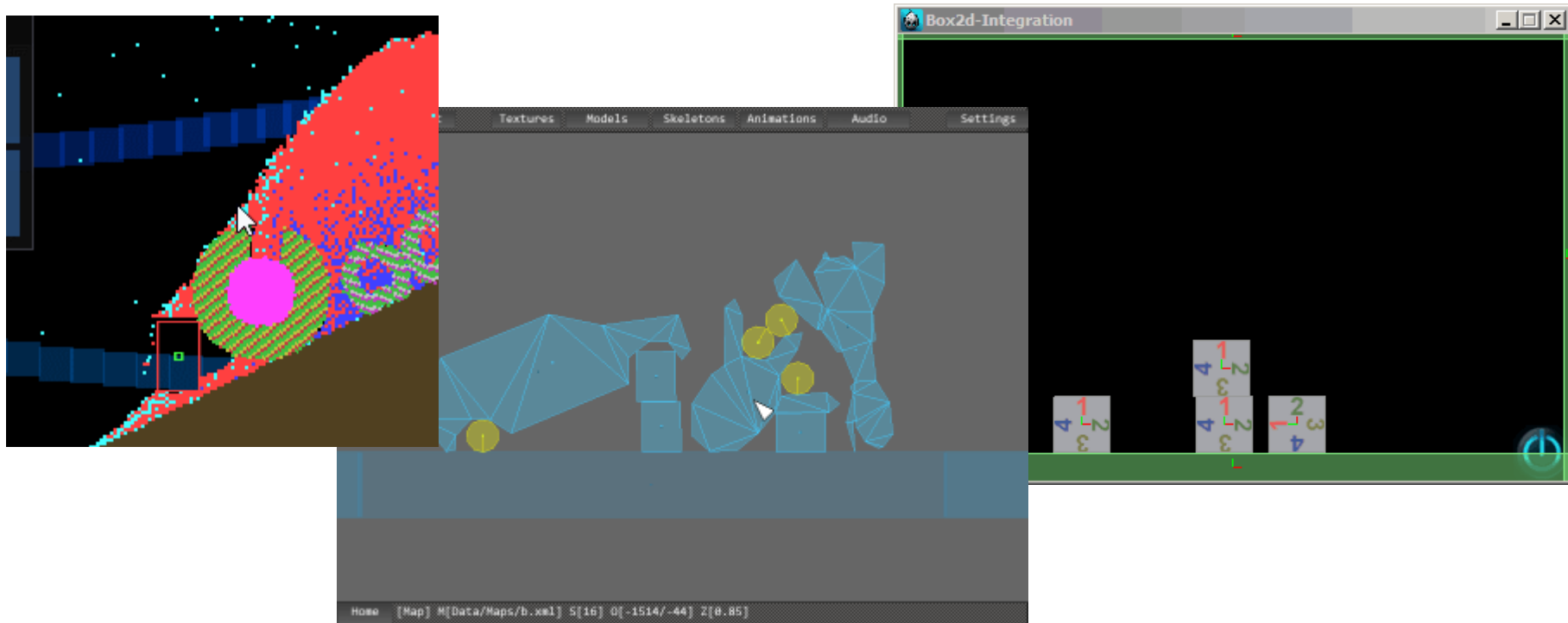
Collisions in Games

- Collision programming involves 2 stages
- Collision **Detection**
 - Given two moving entities with some given shape / bounds, determine if they intersect
 - Detection is a geometric problem
- Collision **Resolution**
 - Given that two entities have collided, determine what to do to 'resolve' the physics of the simulation

Assumptions for Slides

- We will use the course definition of 'Entity'
- Assume that Entities have the following
 - Position (x,y)
 - Bounding Shape (circle, rectangle, lines)
 - Velocity (dx, dy)
- Our goal is to **learn the mathematics** that allows entities to interact in our game

Existing Physics Engines

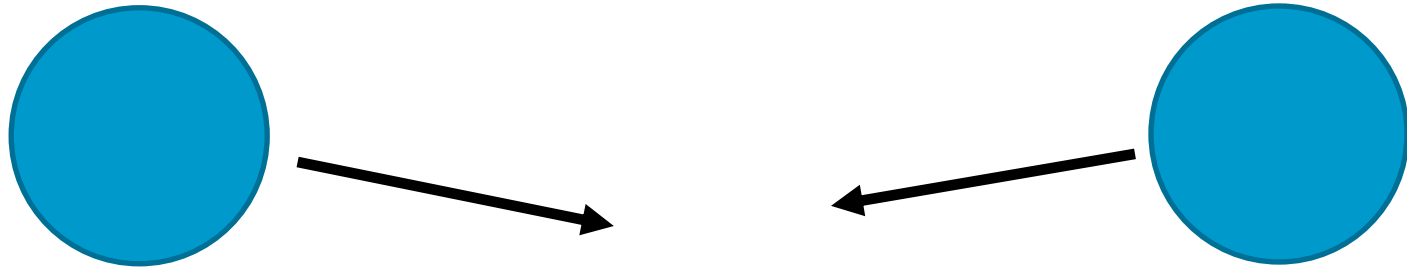


Collision Detection Problems

- Two main ways to handle collisions
 1. Will a collision happen **in the future**?
Where and when will it occur?
 2. Is there a collision happening **right now**?
Compute overlap and resolve

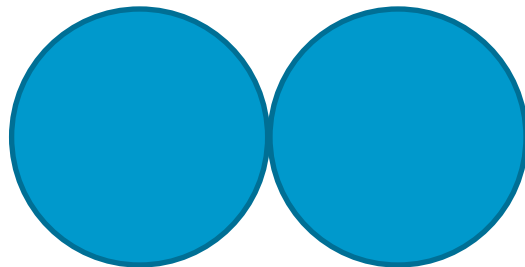
Future Collision Prediction

- Given two entities which have a speed, position, and bounding shape, determine if, when, where they will collide (intersect)



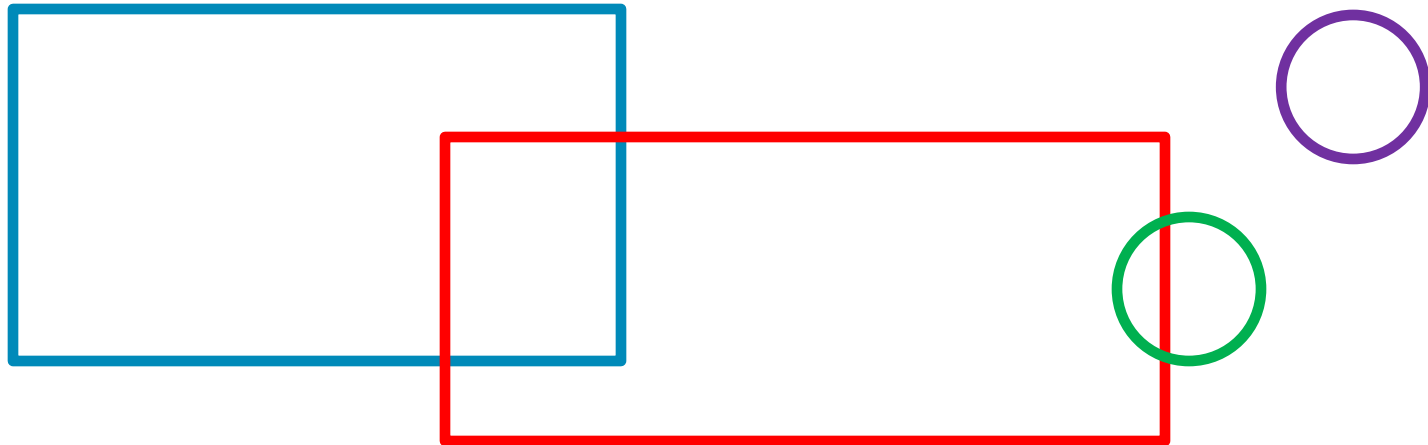
Future Collision Prediction

- Given two entities which have a speed, position, and bounding shape, determine if, when, where they will collide (intersect)



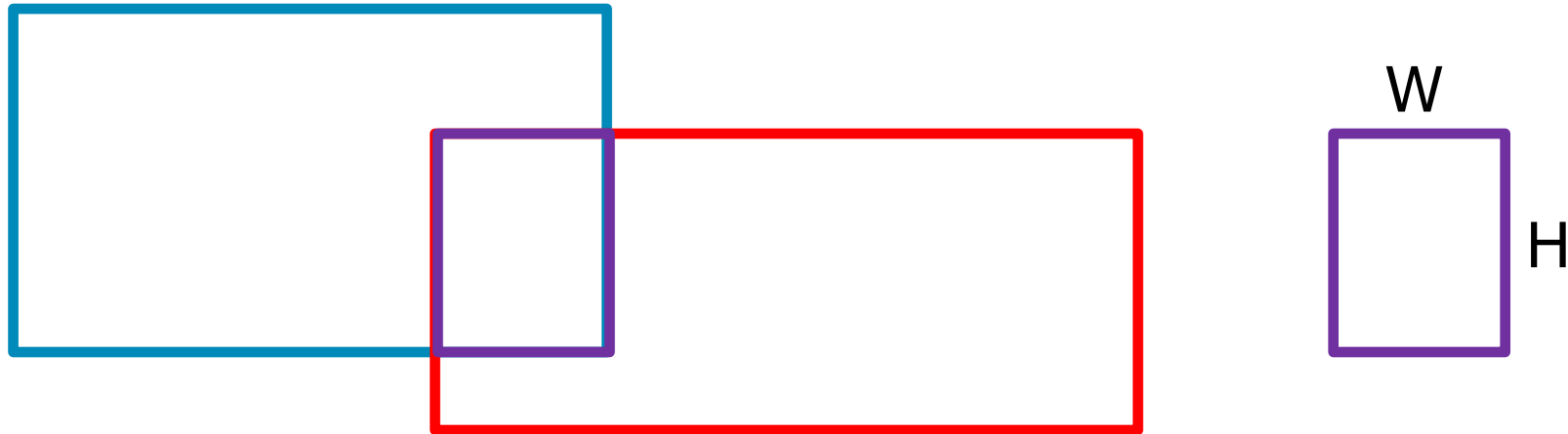
Current Collision Detection

- Given two entities which have a current position (only), do they intersect? (bool)



Current Collision Overlap

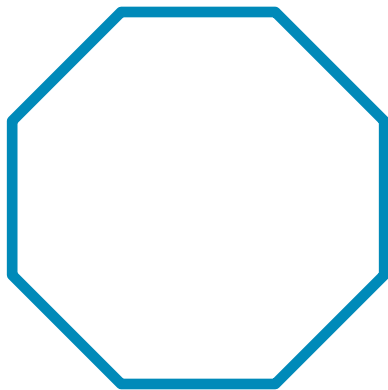
- Given two entities which have a current position (only), what is the overlap? (x,y)



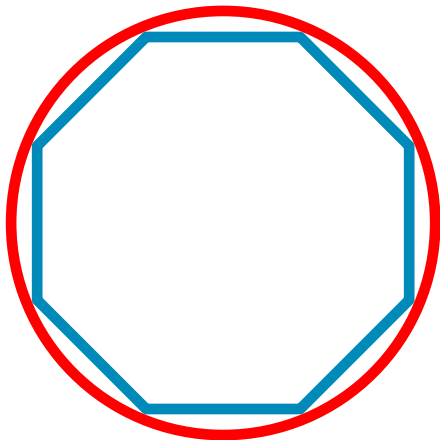
Entity Bounding Shapes

- In the real world, objects can have arbitrary shapes / interaction **surfaces**
- These are **difficult** to store, compute, and simulate accurately / continuously
- All shapes in computer simulations must be made of **primitive** types
 - 2D: Line, Triangle, Circle, Rectangle
 - 3D: Sphere, Plane, Cylinder, Prism
- Bounding shape concept: simulate the **interaction bounds** of a complex shape with a simpler one

Entity Bounding Shape Examples



Entity Bounding Shape Examples



2D Bounding Shapes

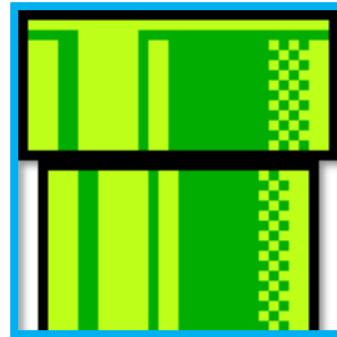
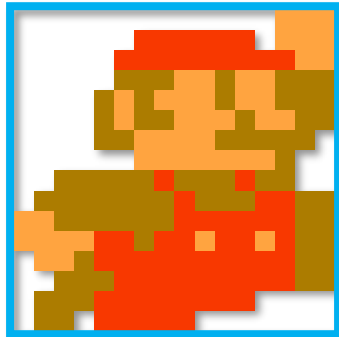
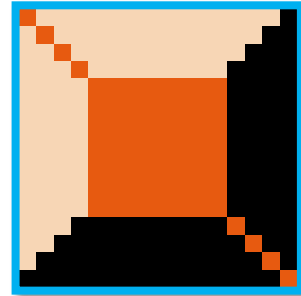
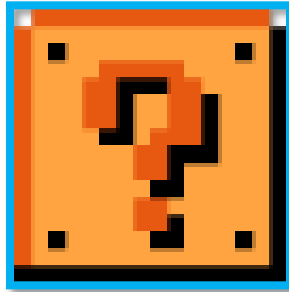
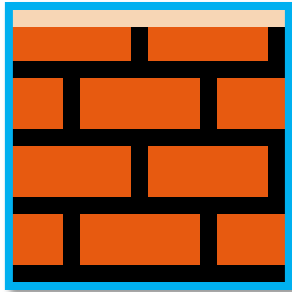
- The **simplest** 2D bounding shape to store and compute is a **circle** with given radius
 - We have already covered circle intersection
- Other 2D bounding shapes can be constructed with a number of **line segments**
- The simplest of these is a **rectangle**
 - Bounding Rectangle / Bounding Box

Bounding Box Size

- Most classic 2D games use **rectangular** sprite graphics
- These rectangular images make a natural fit for rectangular **bounding box**
- Usually, the bounding box will be the **smallest rectangle** that fits the sprite, so we can use the texture size as the bounding box size
- However **sometimes**, this is not the case

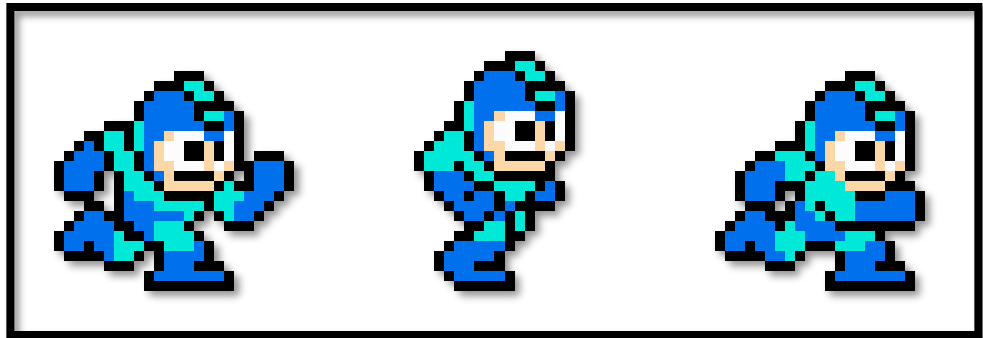
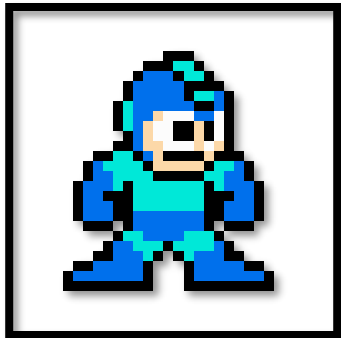


Bounding Box Size



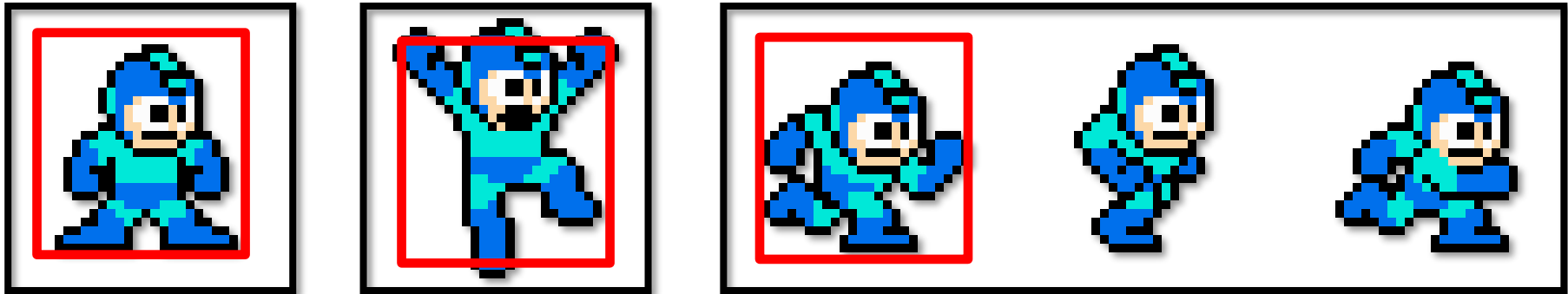
Bounding Box Size

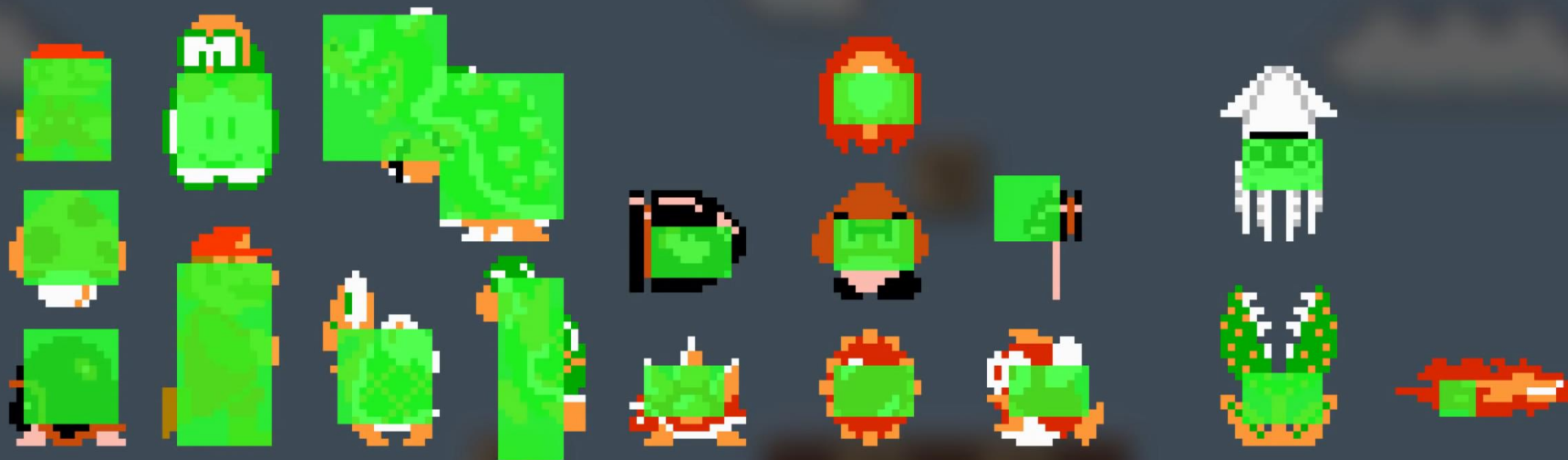
- Sometimes the bounding box does not fit the exact bounds of the sprite texture



Bounding Box Size

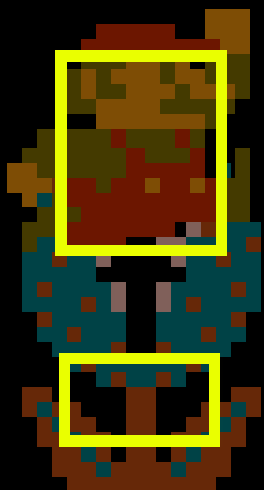
- Sometimes the bounding box does not fit the exact bounds of the sprite texture
- So make a custom sized bounding box





Normal

Weird



MARIO
001000

1x00

WORLD
1-1

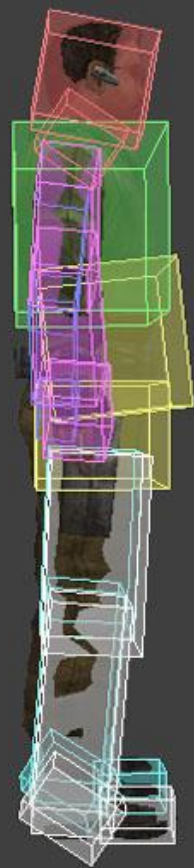
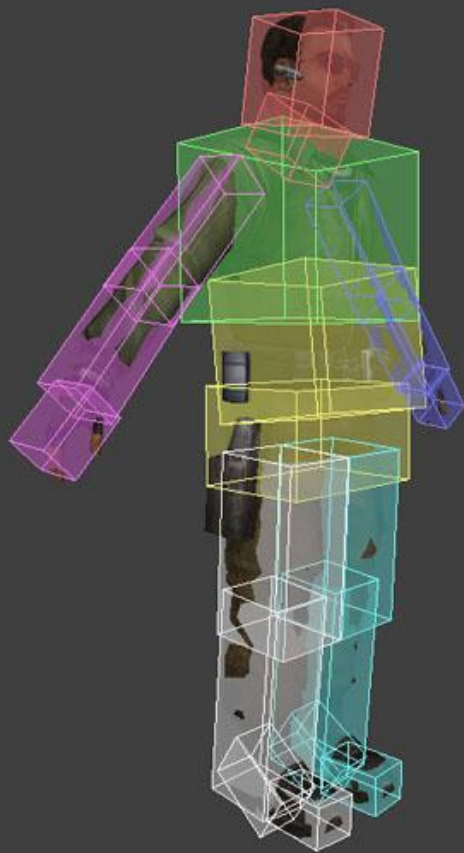
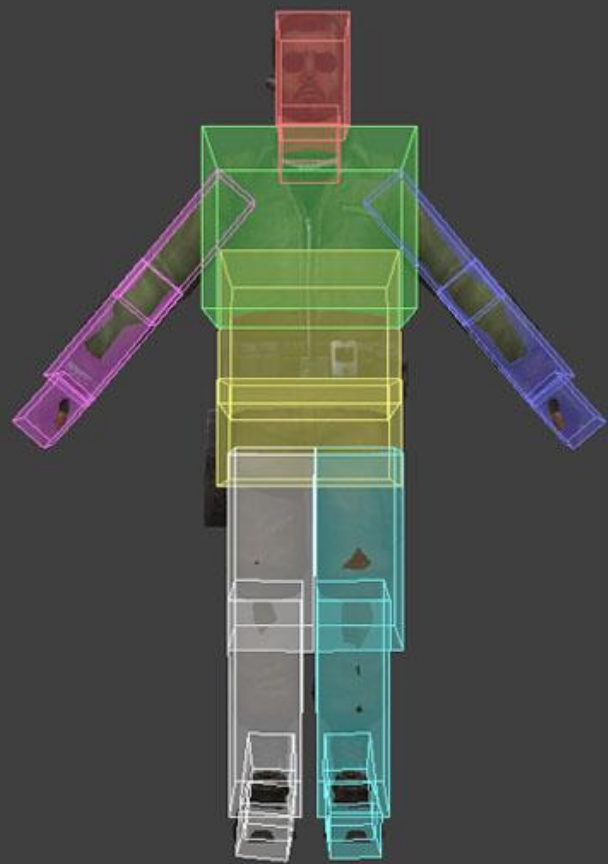
TIME
001



her width is 10 pixels and her height is as follows:

38 14 32 42 20 24 38 38

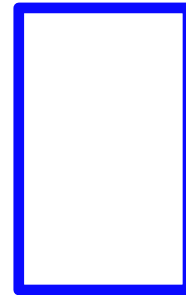
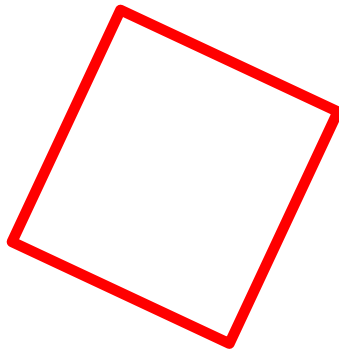




CS:S

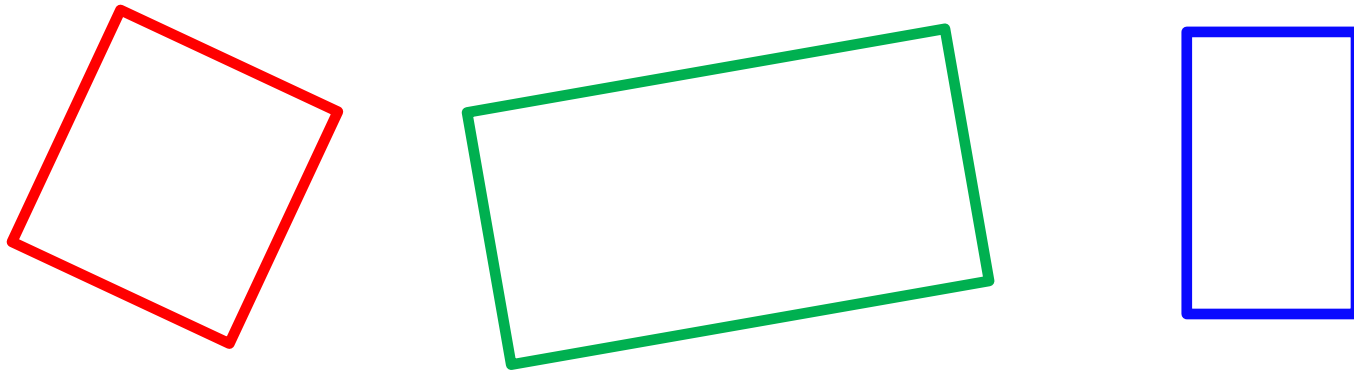
Axis Aligned Bounding Boxes

- Rectangles in a 2D plane can take **any orientation** as long as all 4 corners form 90 degree angles (def of rectangle)



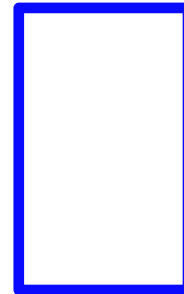
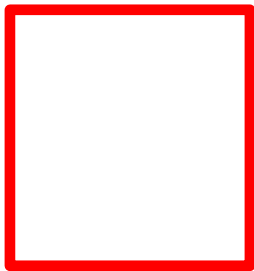
Axis Aligned Bounding Boxes

- Detecting intersections of arbitrary rectangles can be quite **slow**, involving computing if all lines intersect



Axis Aligned Bounding Boxes

- By aligning our rectangles with the x / y axis, we can trade arbitrary alignment for very fast intersection / collision checks
- Axis Aligned Bounding Box = **AABB**



Point Inside AABB

- The simplest collision check we can do is to see if a single point is inside an AABB

x_1, y_1

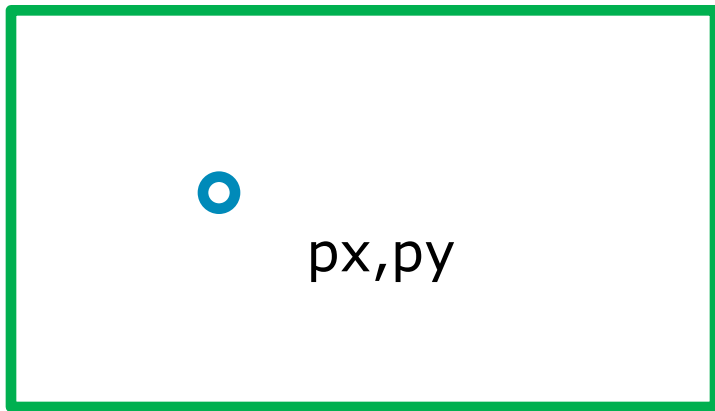


x_2, y_2



px_1, py_1

Point Inside AABB

 x_1, y_1  px, py x_2, y_2

P inside if

$$px > x_1$$

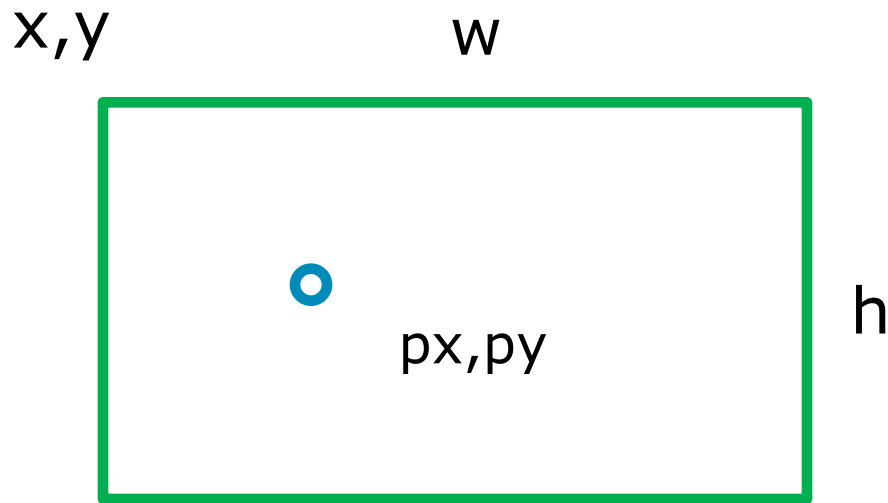
$$px < x_2$$

$$py > y_1$$

$$py < y_2$$

(0,0) top-left

Point Inside AABB



P inside if

$$p_x > x$$

$$p_x < x + w$$

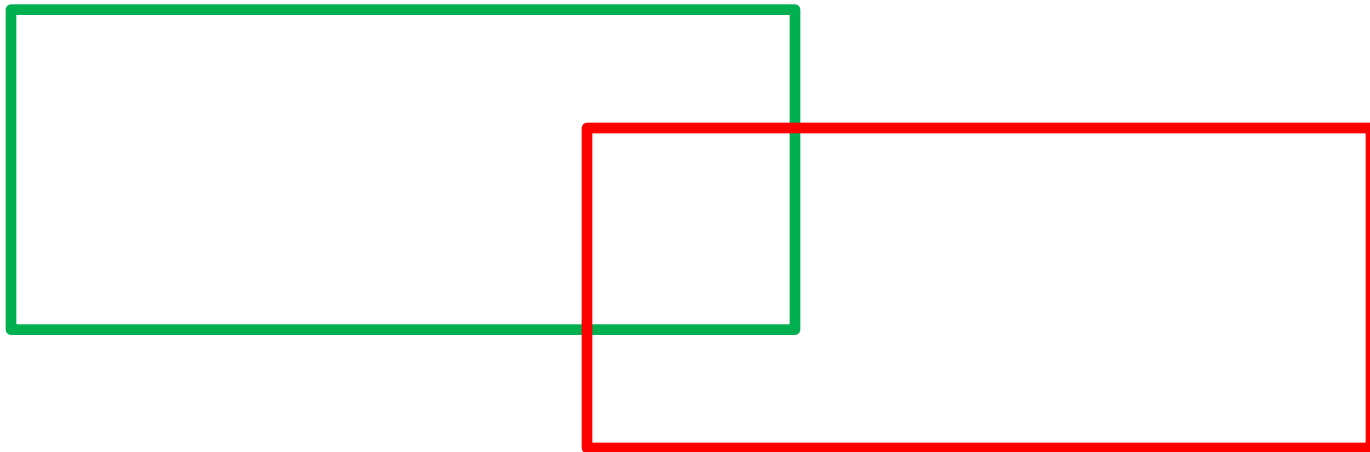
$$p_y > y$$

$$p_y < y + h$$

(0,0) top-left

AABB Intersection

- Next step is to determine if two AABB intersect anywhere



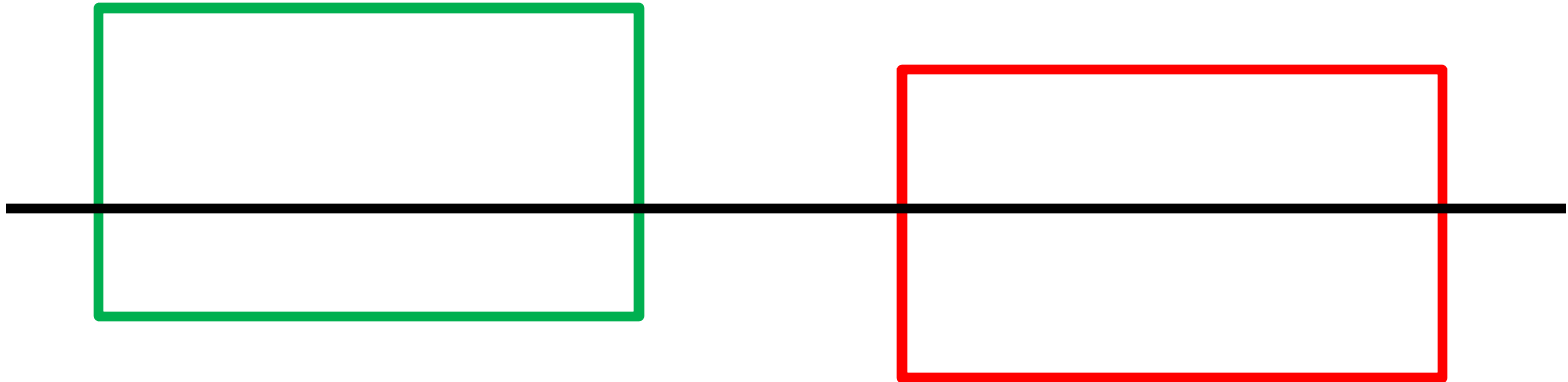
AABB Horizontal Overlap

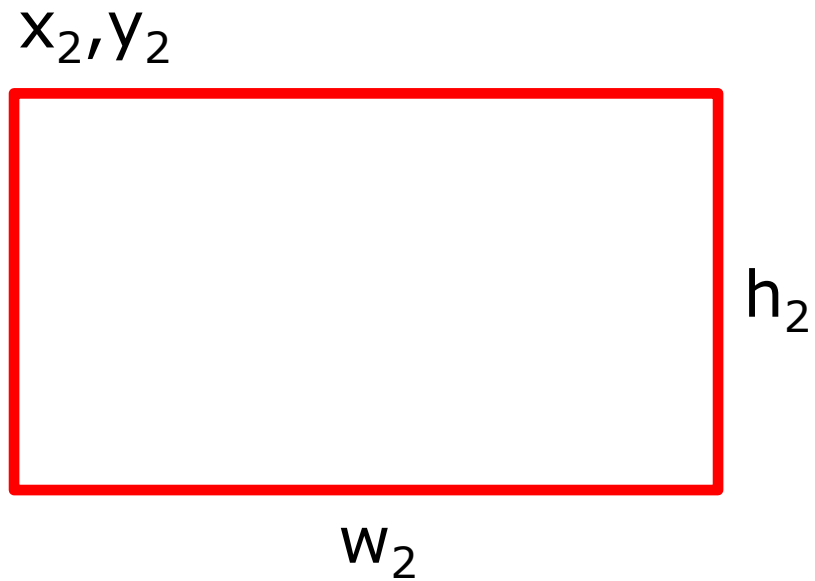
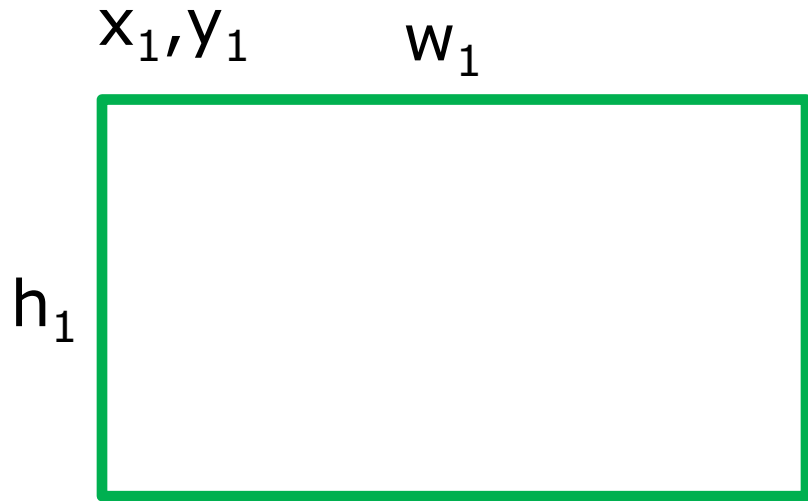
- Horizontal overlap occurs if the top of each box is higher than the bottom of the other

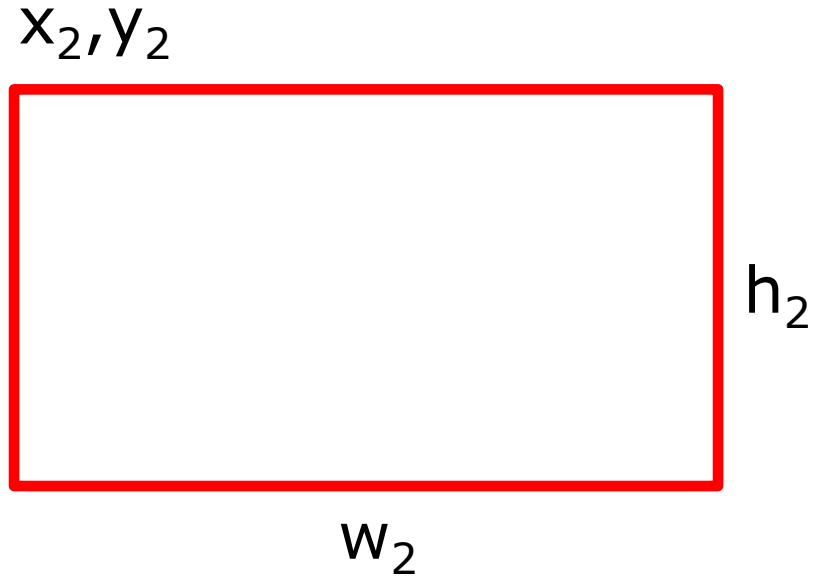
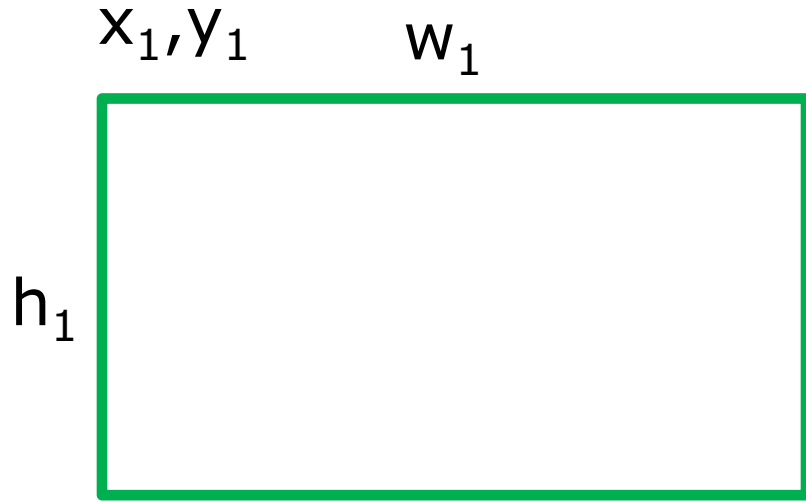


AABB Horizontal Overlap

- Horizontal overlap occurs if the top of each box is higher than the bottom of the other







Horizontal Overlap if

$$y_1 < y_2 + h_2$$

$$y_2 < y_1 + h_1$$

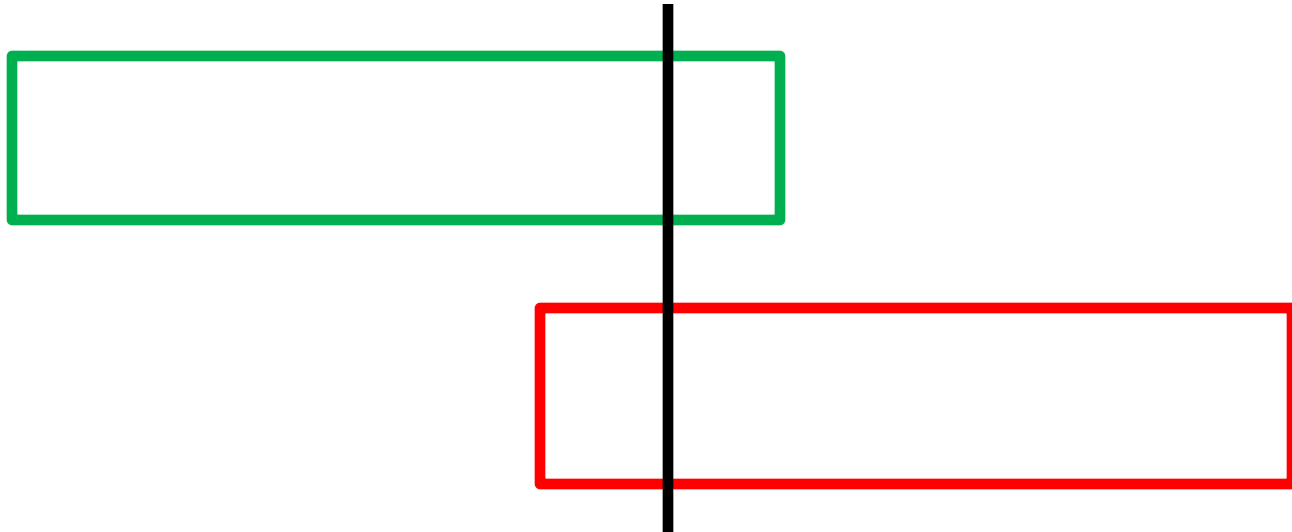
AABB Vertical Overlap

- Vertical overlap occurs if the left of each box is less than the right of the other



AABB Vertical Overlap

- Vertical overlap occurs if the left of each box is less than the right of the other



x_1, y_1

w_1



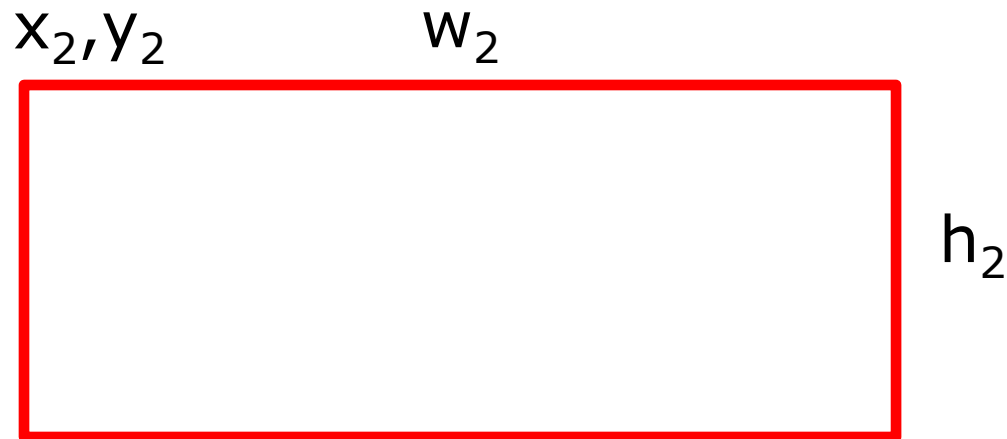
h_1

x_2, y_2

w_2



h_2



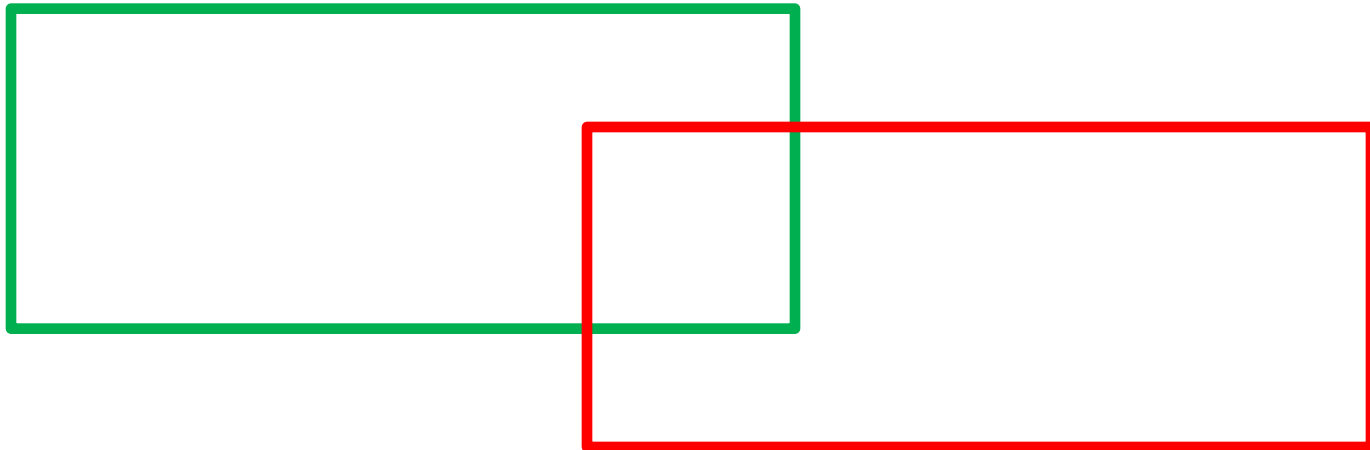
Vertical Overlap if

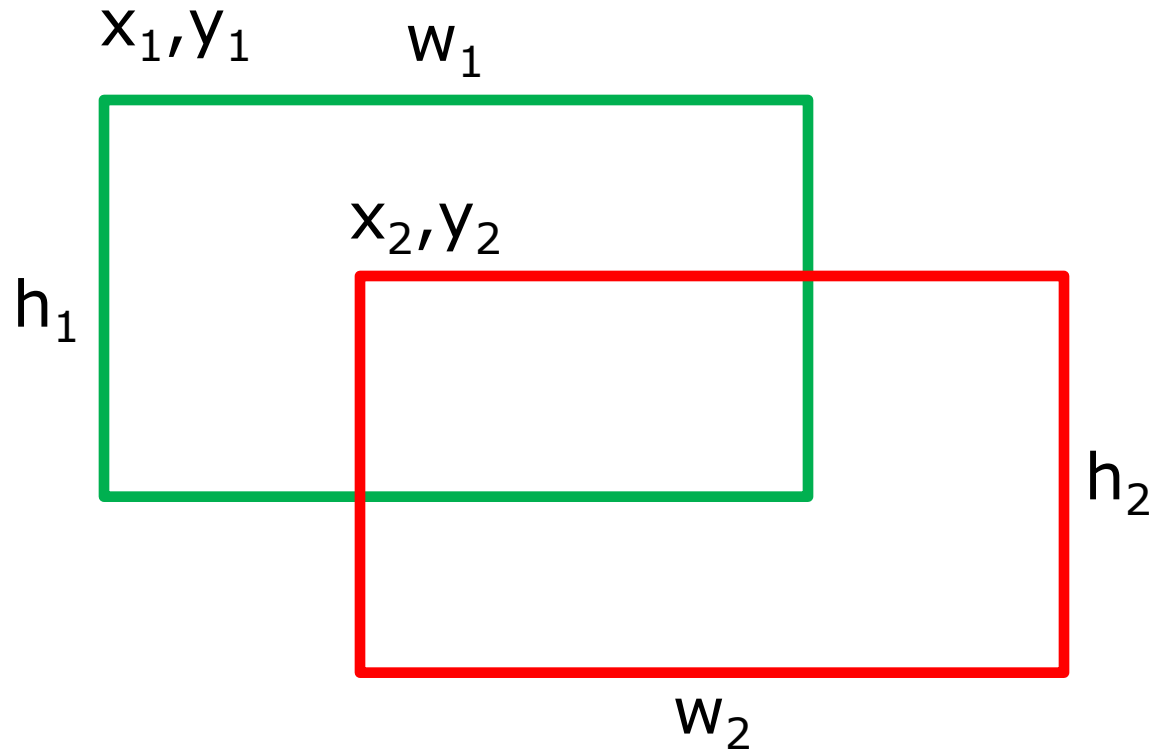
$$x_1 < x_2 + w_2$$

$$x_2 < x_1 + w_1$$

AABB Intersection

- Two AABB intersect if they overlap both horizontally and vertically





AABB Intersect if

$$x_1 < x_2 + w_2$$

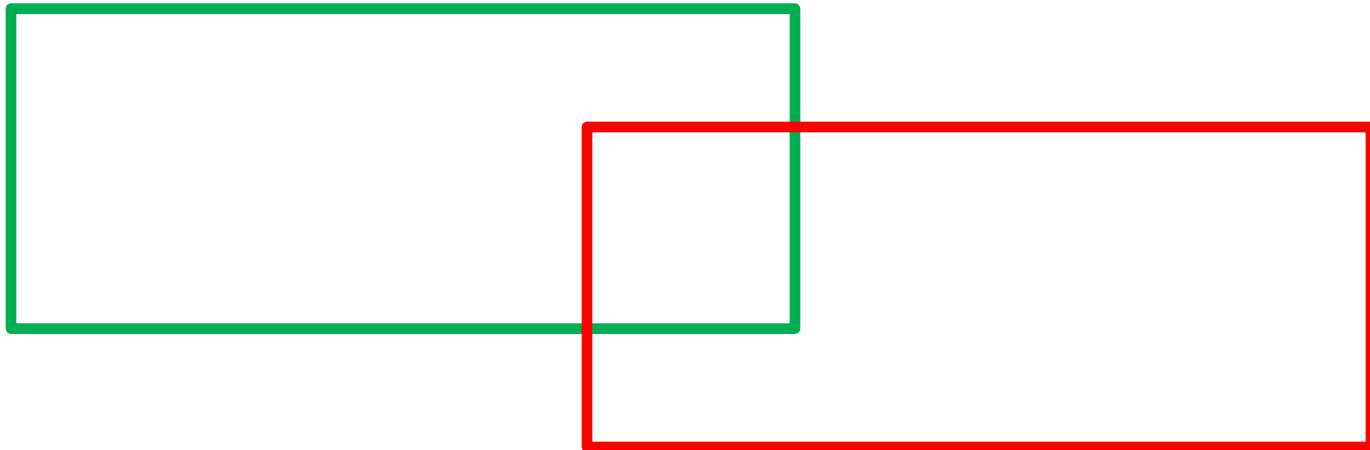
$$x_2 < x_1 + w_1$$

$$y_1 < y_2 + h_2$$

$$y_2 < y_1 + h_1$$

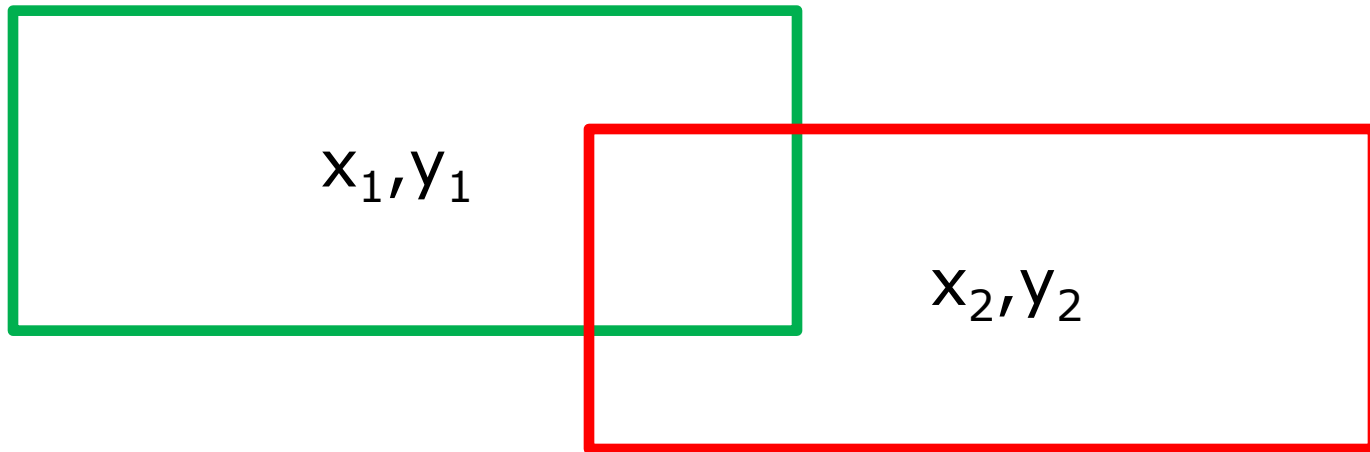
AABB Intersection

- This tells us IF two AABB intersect, but does not tell us by how much



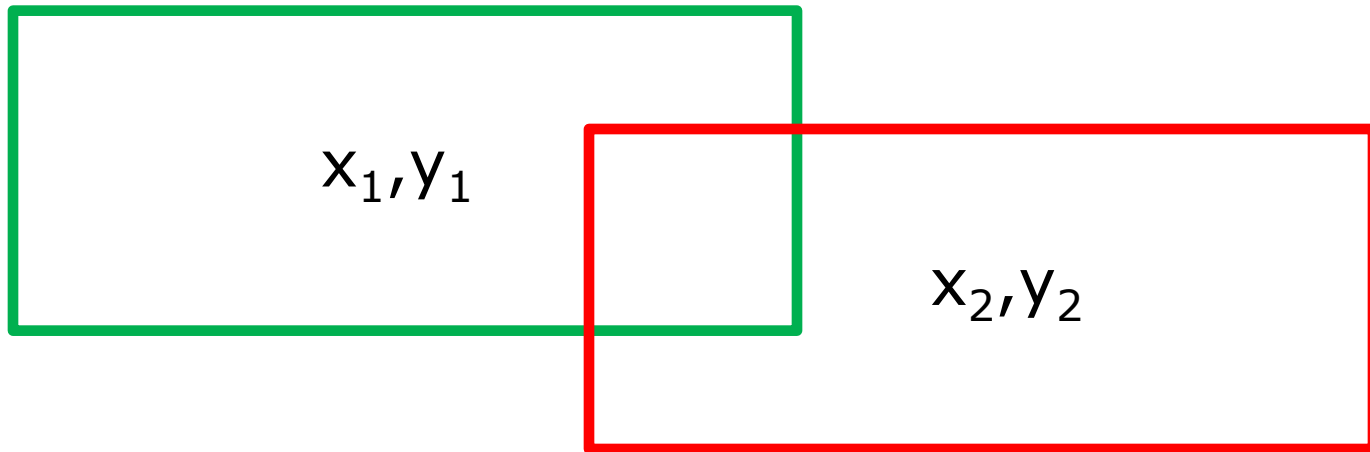
Centered AABB Intersection

- Our game engine uses the CENTER of the entity as the position for the AABB

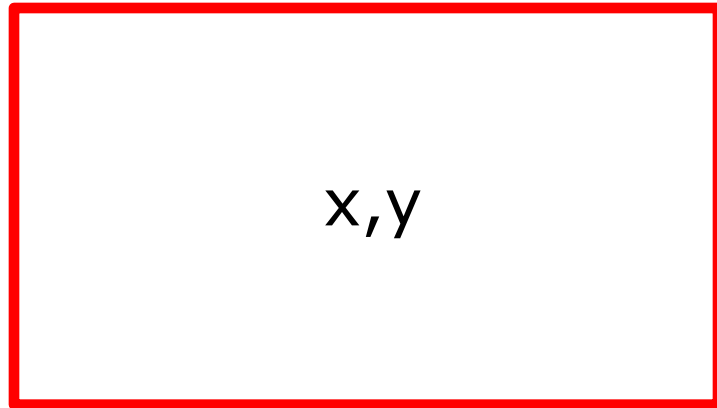


Centered AABB Overlap

- How do we calculate the overlapping area for center positioned AABB?



w



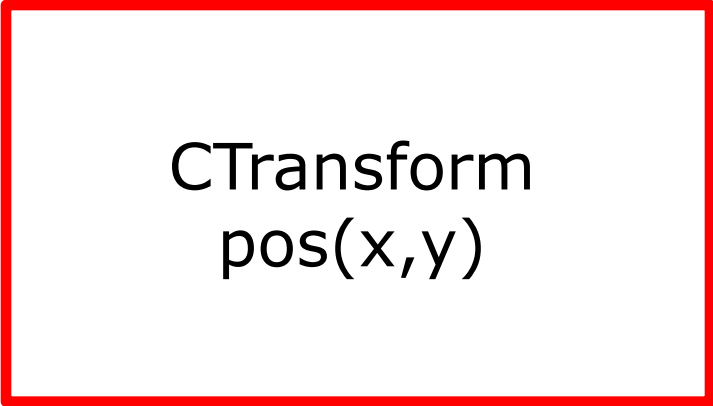
x, y

h

```
class CTransform : public Component
{
public:
    Vec2 pos          = { 0.0, 0.0 };
    Vec2 prevPos      = { 0.0, 0.0 };
    Vec2 scale        = { 1.0, 1.0 };
    Vec2 speed        = { 0.0, 0.0 };
    float angle = 0;
```

```
class CBoundingBox : public Component
{
public:
    Vec2 size;
    Vec2 halfSize;
    CBoundingBox(const Vec2 & s)
        : size(s), halfSize(s.x / 2, s.y / 2) {}
};
```

CBBBox size.x



CTransform
pos(x,y)

CBBBox size.y

class CTsanhgong ruclic Congrongentj

ruclic

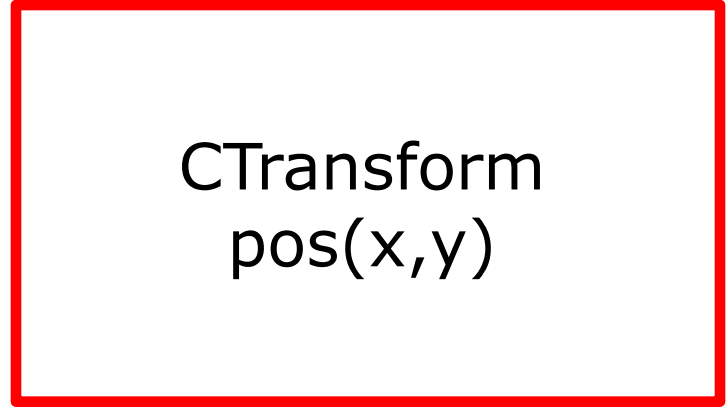
leç,ğ	řôş				
leç,ğ	řsêwRôş	°	°	°	°
leç,ğ	şçălê	°	°	°	°
leç,ğ	wêlôçîţj	°	°	°	°
ğlôăţj	ănglê	°			

class CBoungdingBou ruclic Congrongentj

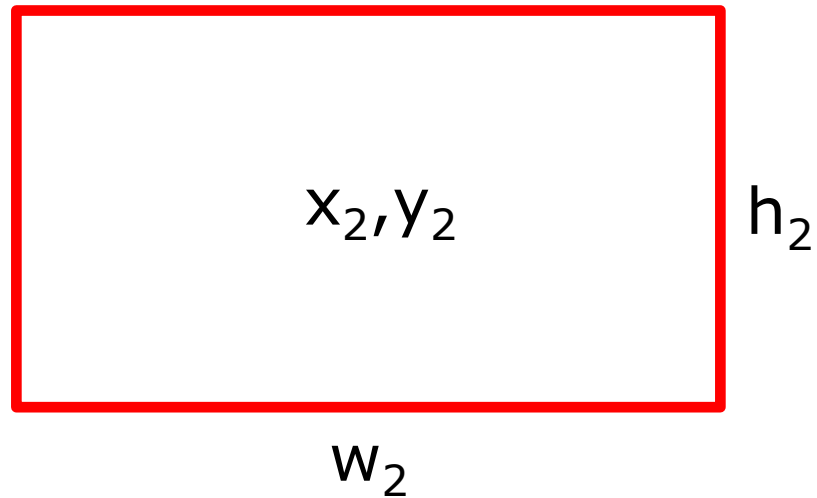
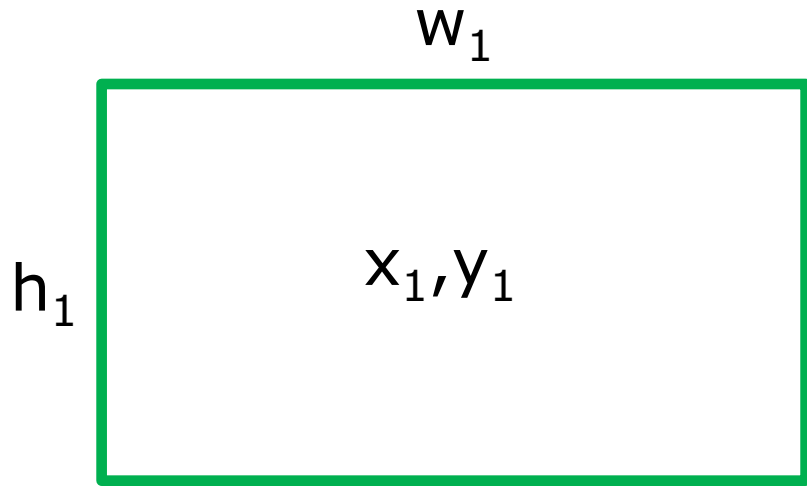
ruclic

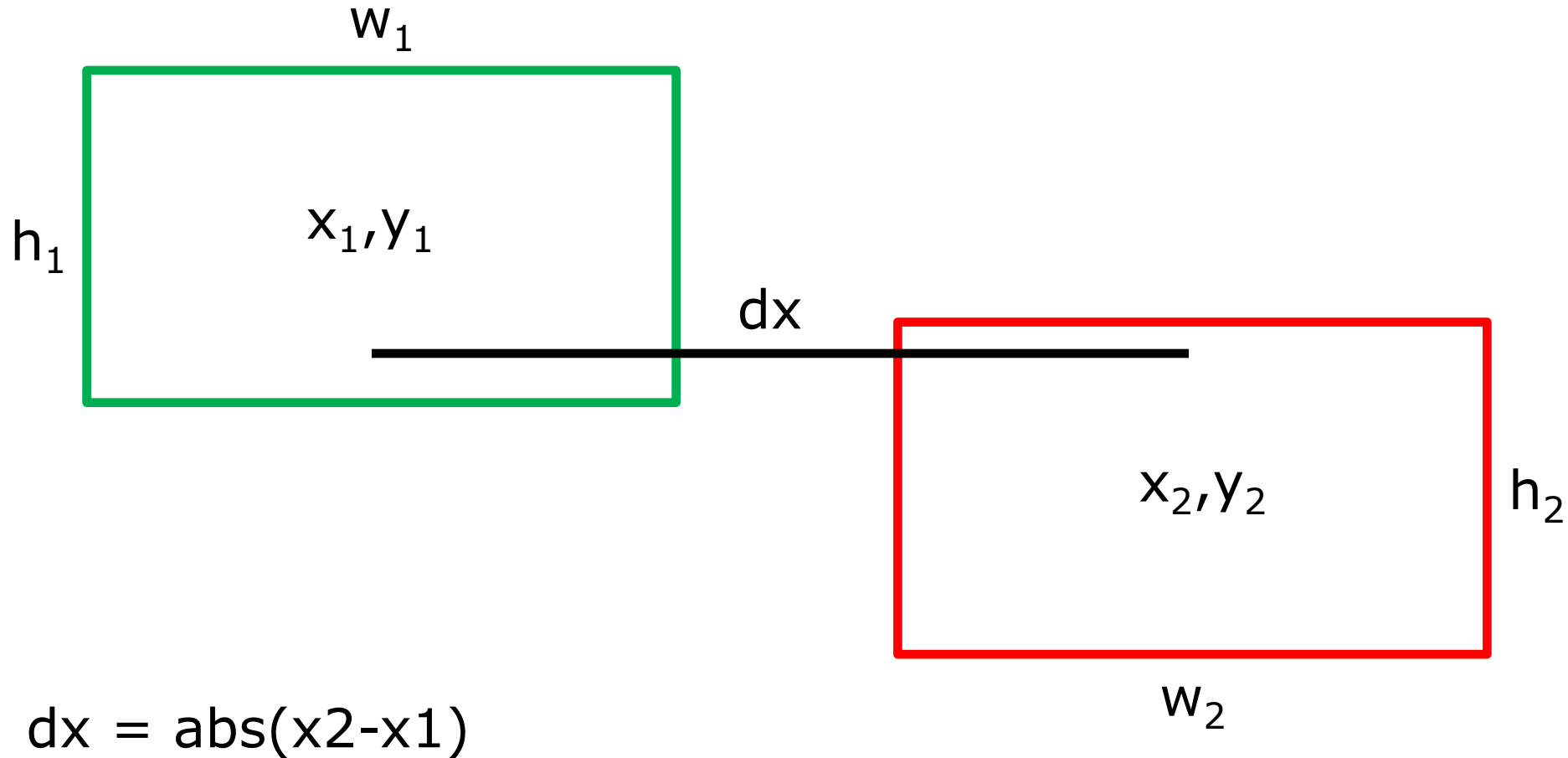
leç,ğ	şîcê				
leç,ğ	hălgşîcê				
CBoungdingBou	đêgău'ţj				
CBoungdingBou	çonşţj leç,ğ	ş			
şîcê	ş	hălgşîcê	ş	y	,
				ş	y

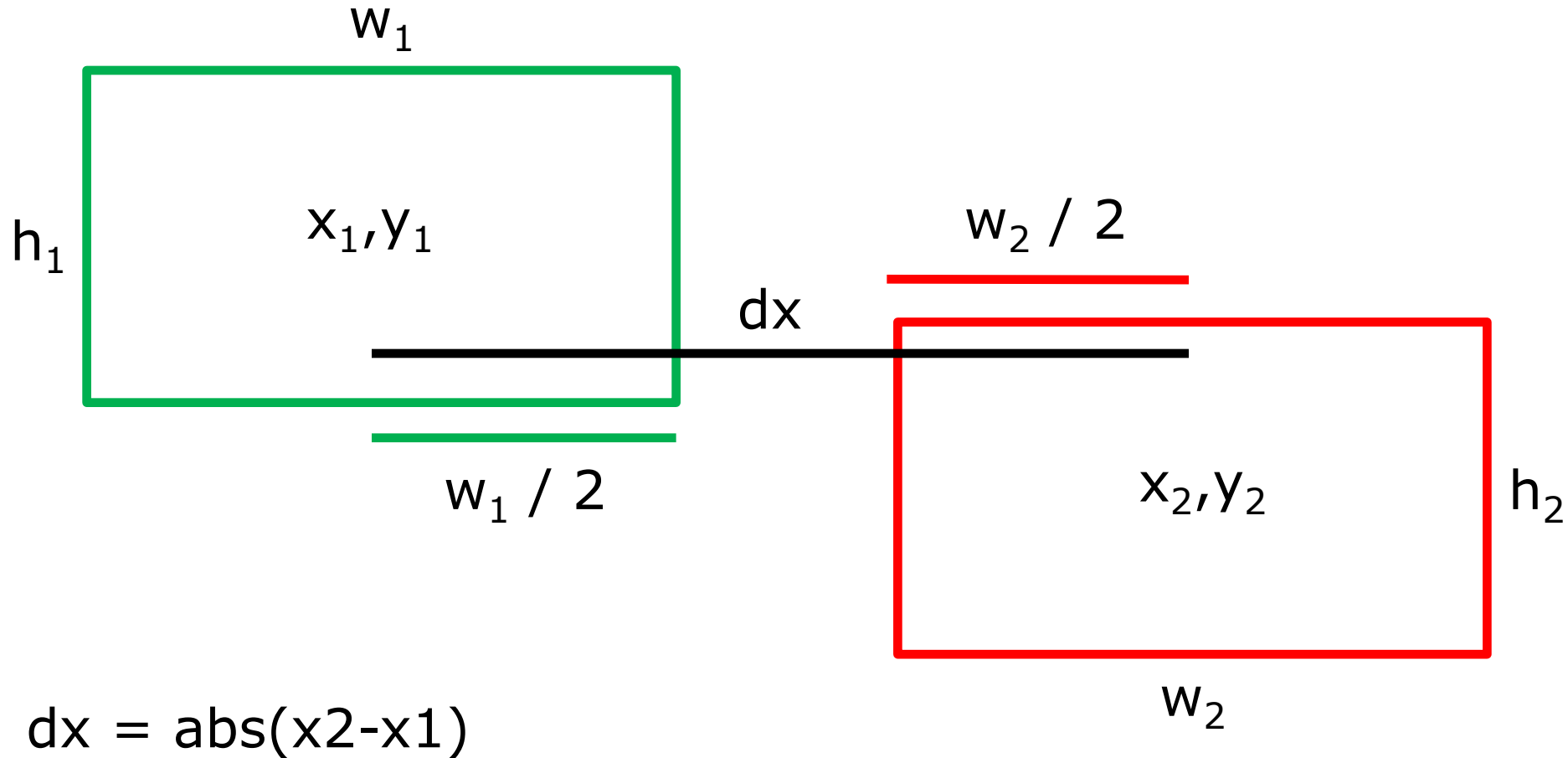
CBox size.x

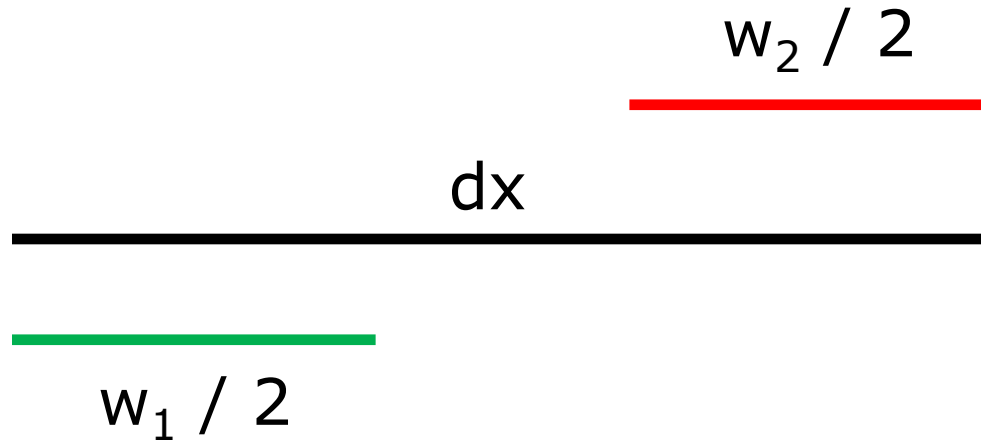


CBox size.y

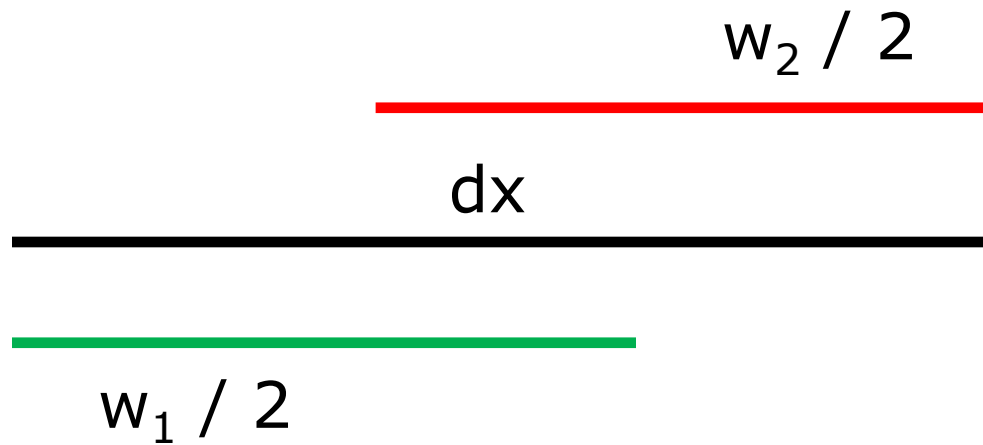




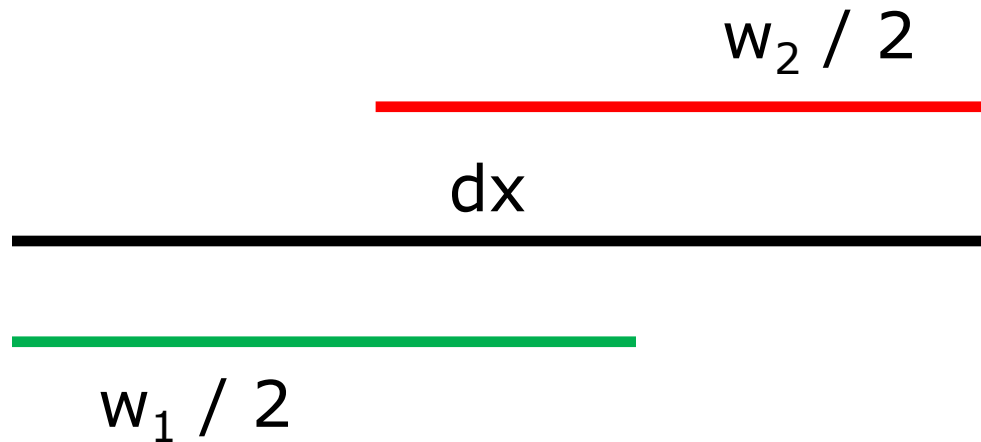




$$dx = \text{abs}(x_2 - x_1)$$



$$dx = \text{abs}(x_2 - x_1)$$

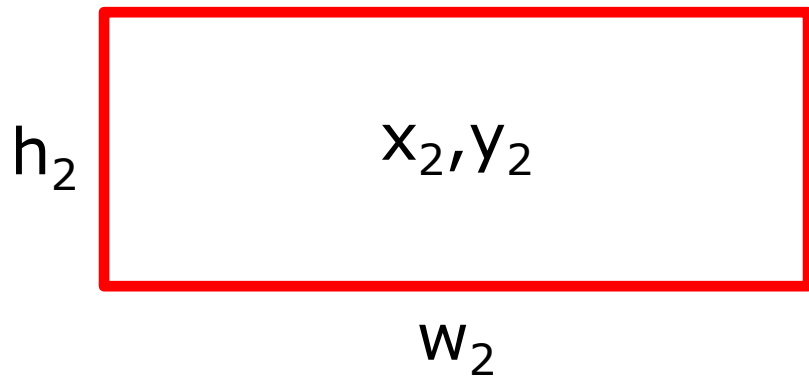
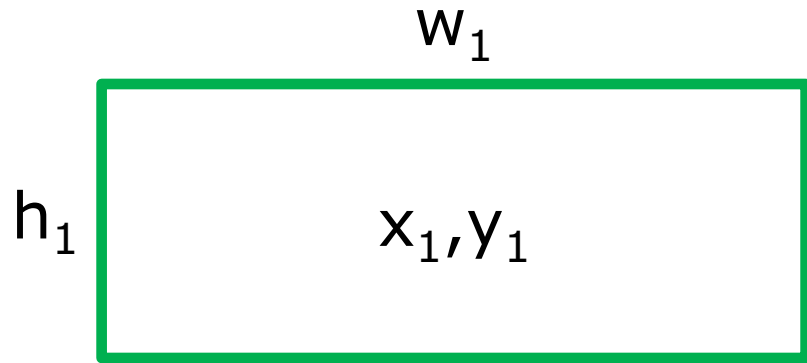


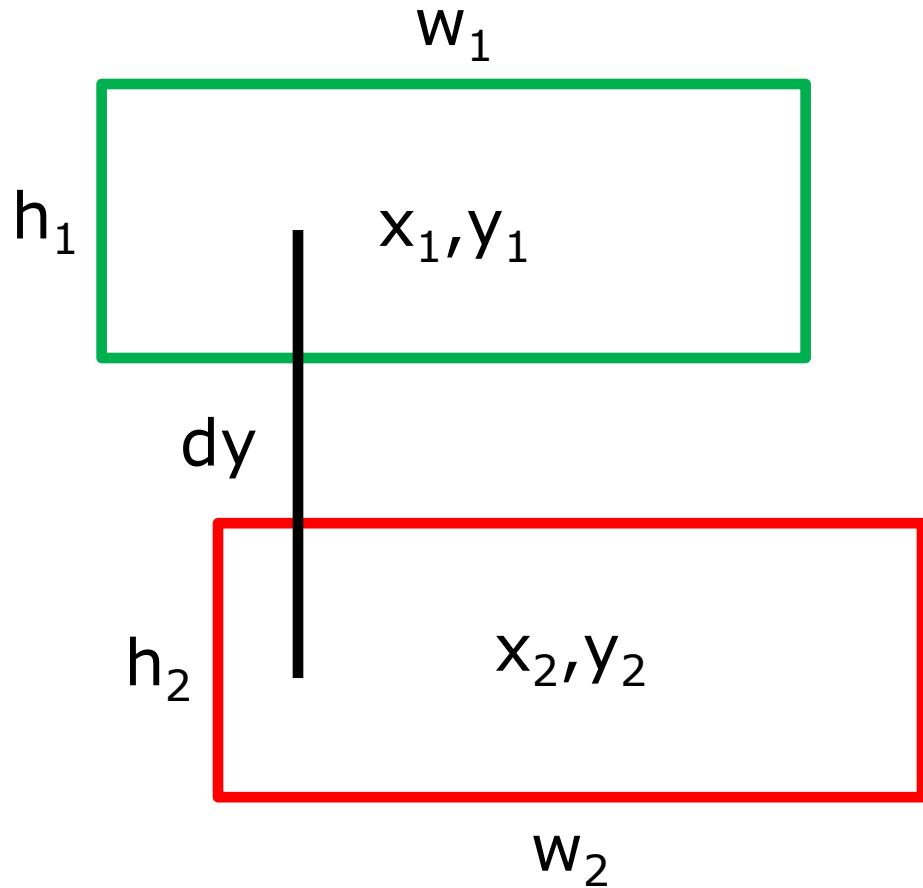
Overlap in x-direction

$$(w_2 / 2) + (w_1 / 2) - dx$$

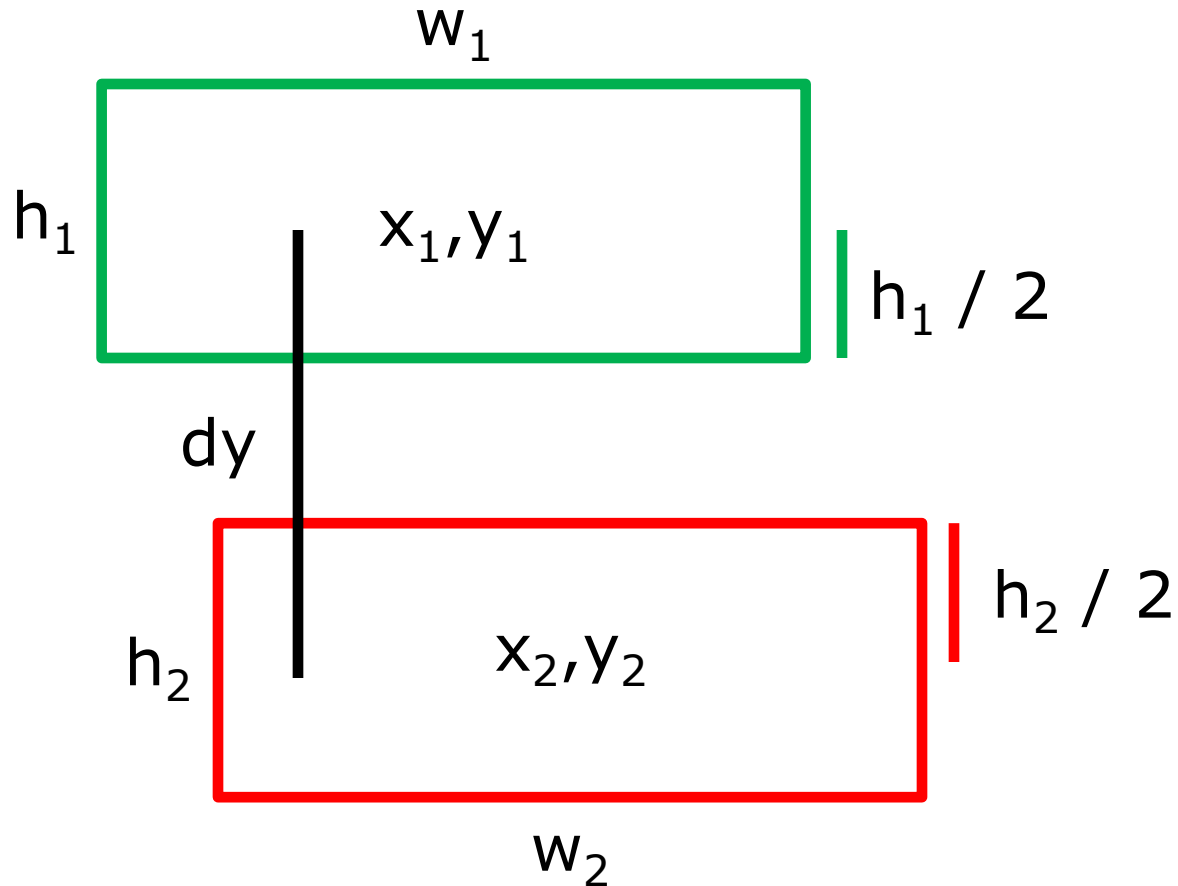
Positive if x overlap

$$dx = \text{abs}(x_2 - x_1)$$



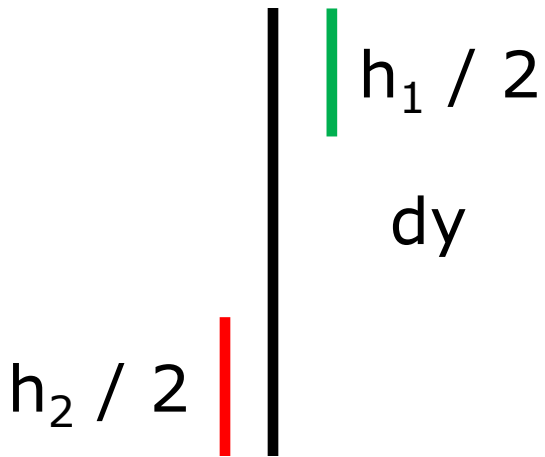


$$dy = \text{abs}(y_2 - y_1)$$

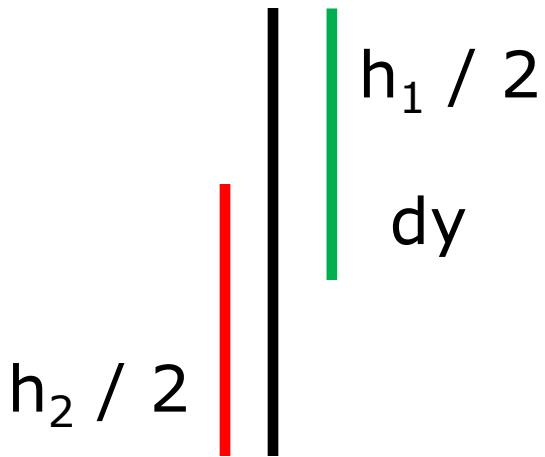


$$dy = \text{abs}(y_2 - y_1)$$

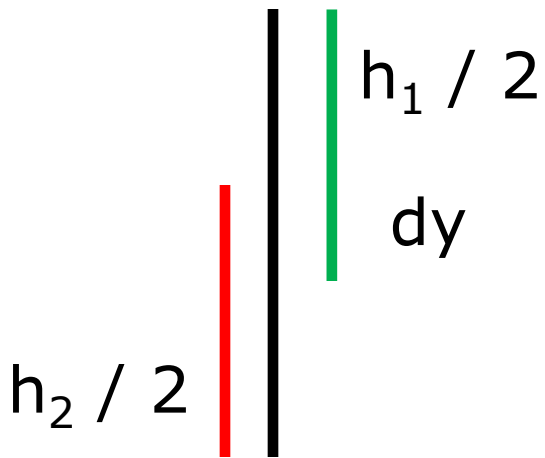
$$dy = \text{abs}(y_2 - y_1)$$



$$dy = \text{abs}(y_2 - y_1)$$



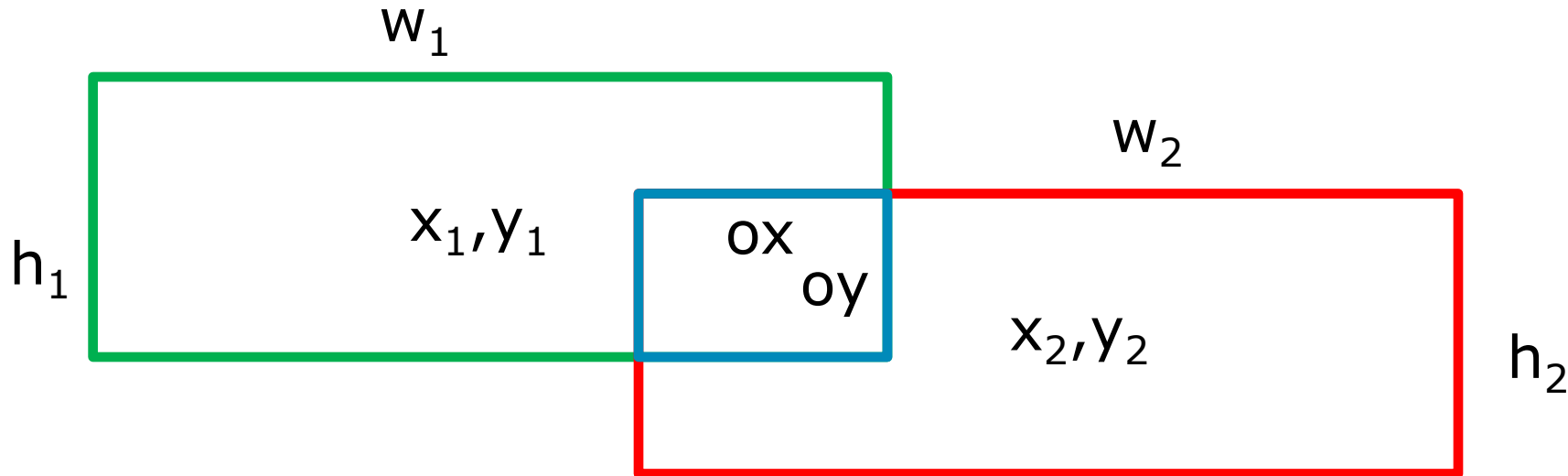
$$dy = \text{abs}(y_2 - y_1)$$



Overlap in y-direction

$$(h_1 / 2) + (h_2 / 2) - dy$$

Positive if y overlap

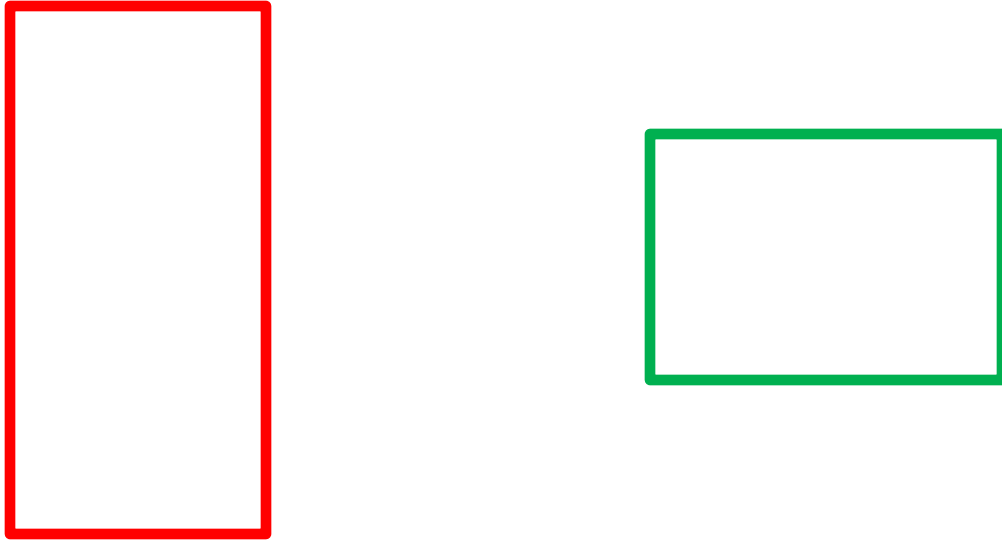


```
delta = [abs(x1-x2), abs(y1-y2)]  
ox = (w1 / 2) + (w2 / 2) - delta.x  
oy = (h1 / 2) + (h2 / 2) - delta.y  
overlap = [ox, oy]
```

Resolving Collisions

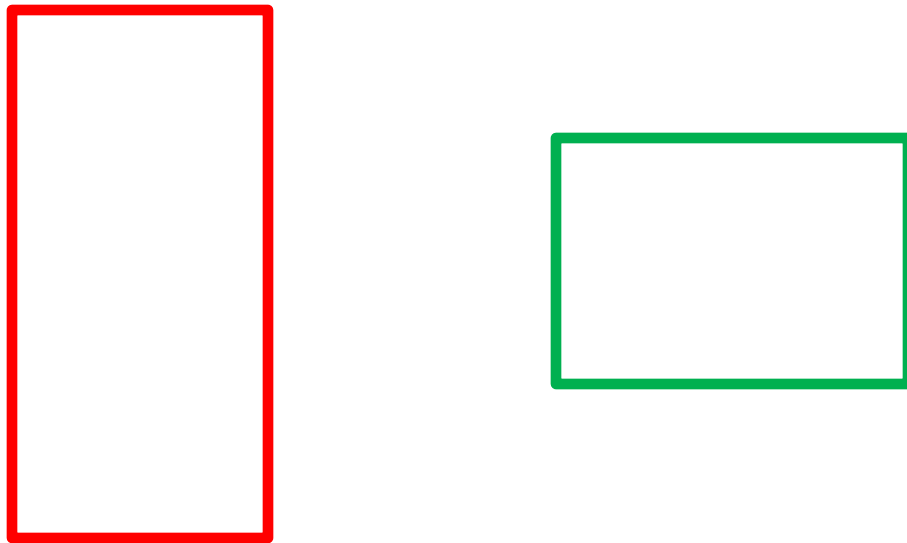
- Now that we have **detected** an AABB collision, what do we do?
- **Resolution** of collisions depends entirely on what the physics of your world should do
- For example: If the player collides with a tile, it should be 'pushed out' to no overlap

Resolving Collisions



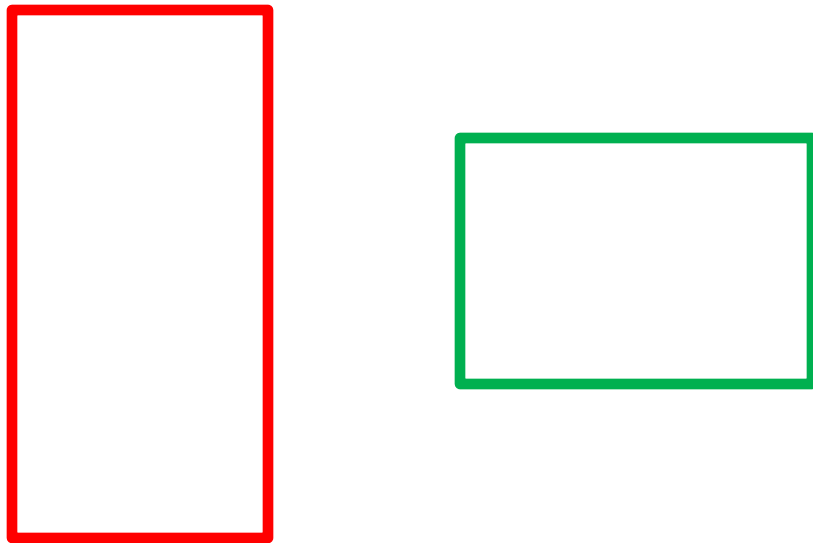
FRAME 1

Resolving Collisions



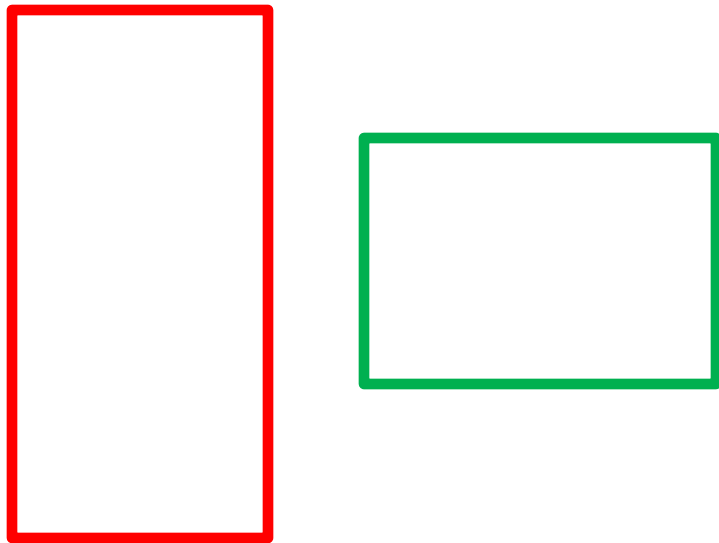
FRAME 2

Resolving Collisions



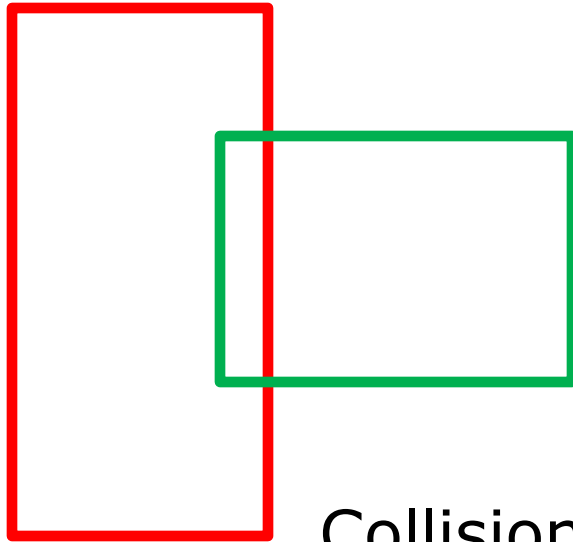
FRAME 3

Resolving Collisions



FRAME 4

Resolving Collisions

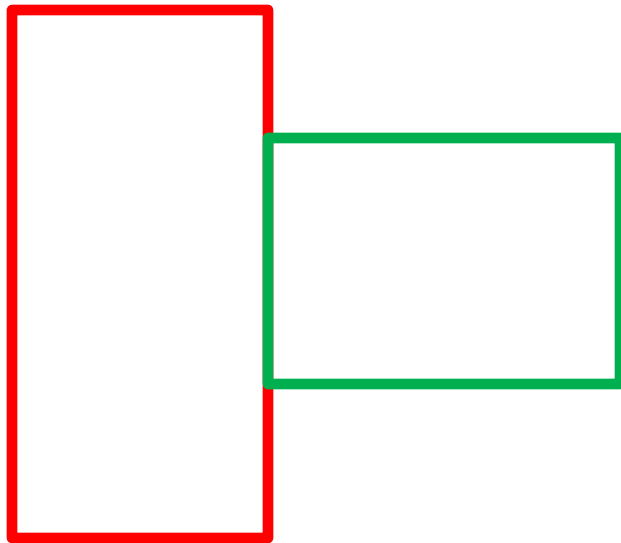


Collision Detected

Resolution: $\text{Green.pos} += \text{Overlap}$

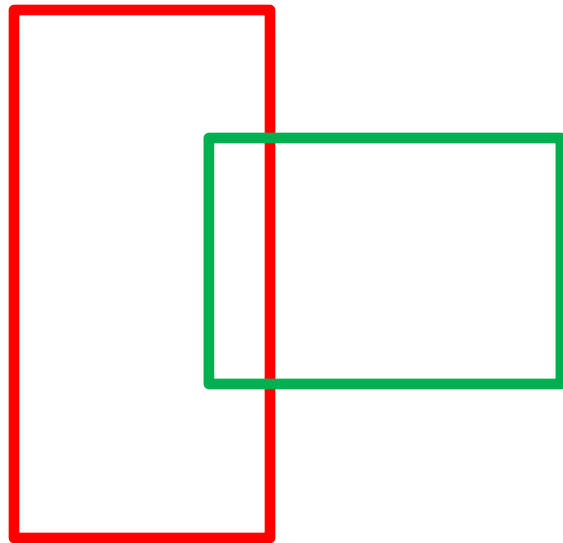
FRAME 5

Resolving Collisions



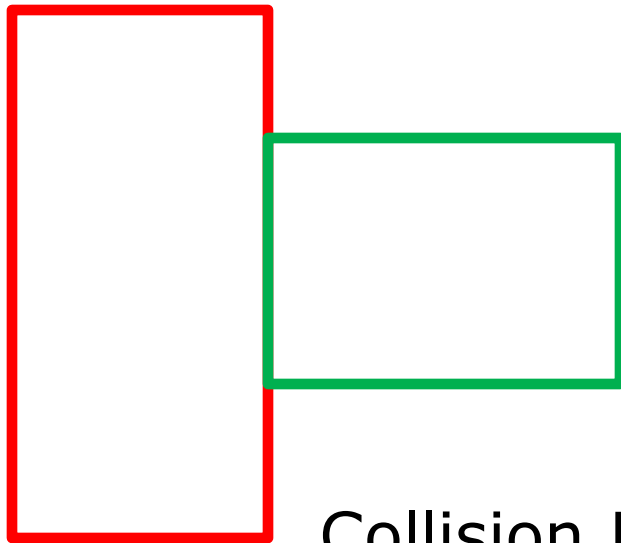
FRAME 5

Resolving Collisions



FRAME 6

Resolving Collisions

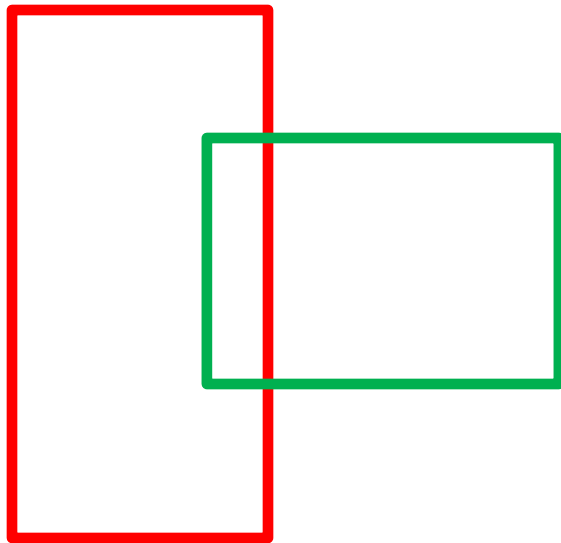


Collision Detected

Resolution: $\text{Green.pos} += \text{Overlap}$

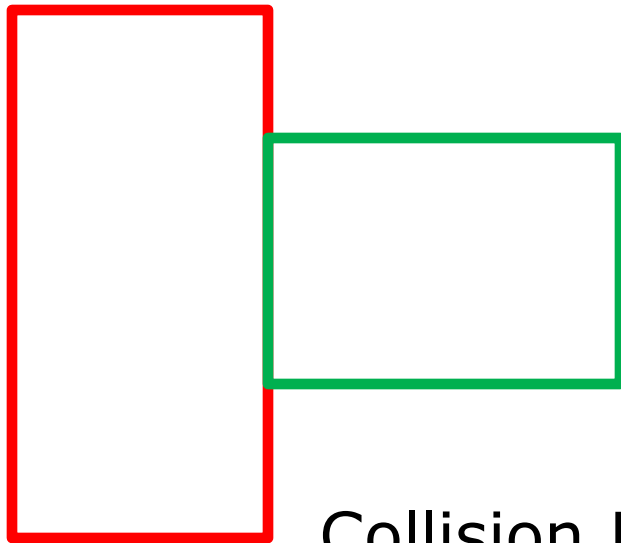
FRAME 6

Resolving Collisions



FRAME 7

Resolving Collisions



Collision Detected

Resolution: $\text{Green.pos} += \text{Overlap}$

FRAME 7

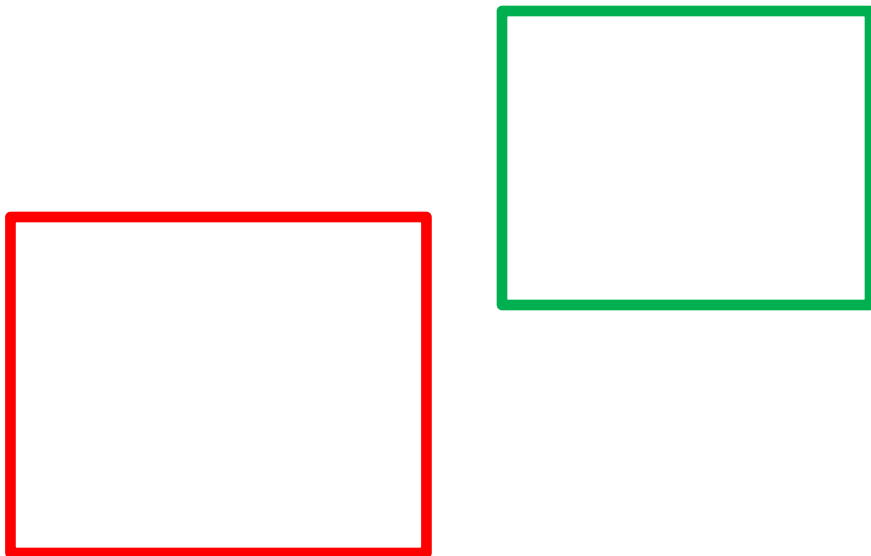
Tricky Resolution Cases



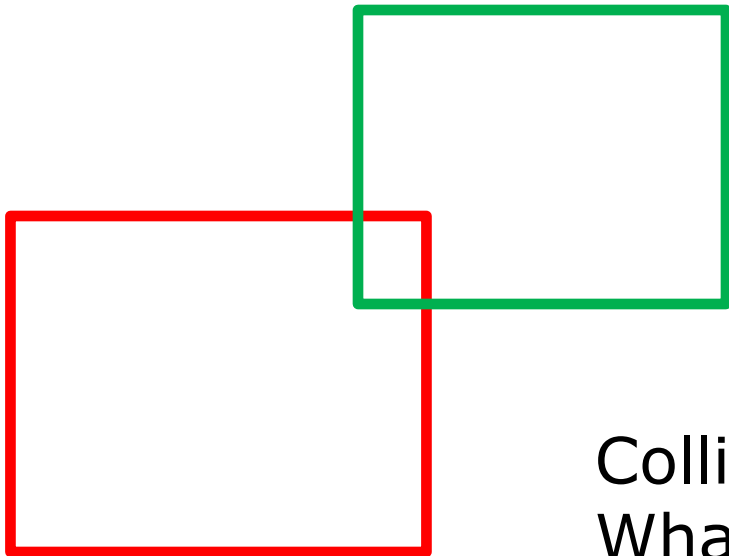
Tricky Resolution Cases



Tricky Resolution Cases

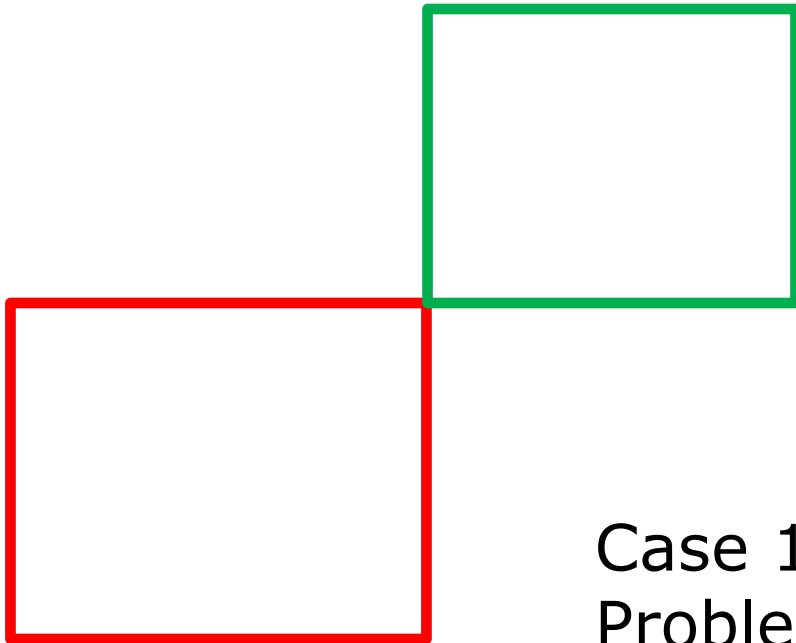


Tricky Resolution Cases



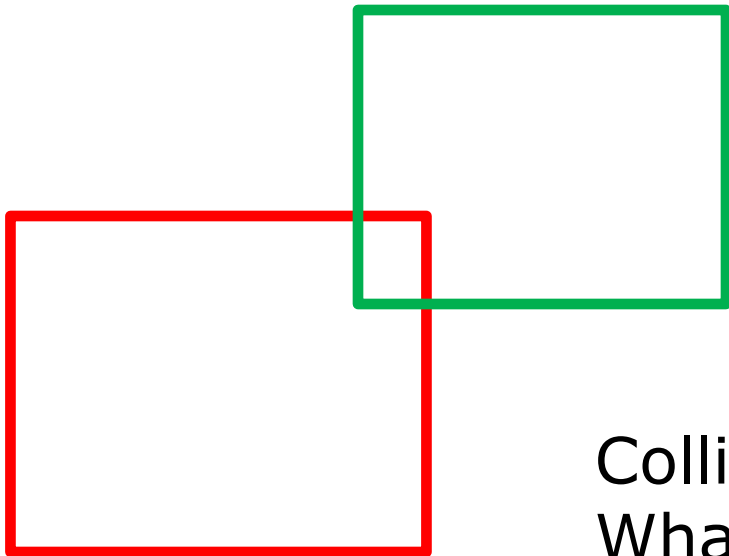
Collision Detected
What to do? Have Options

Tricky Resolution Cases



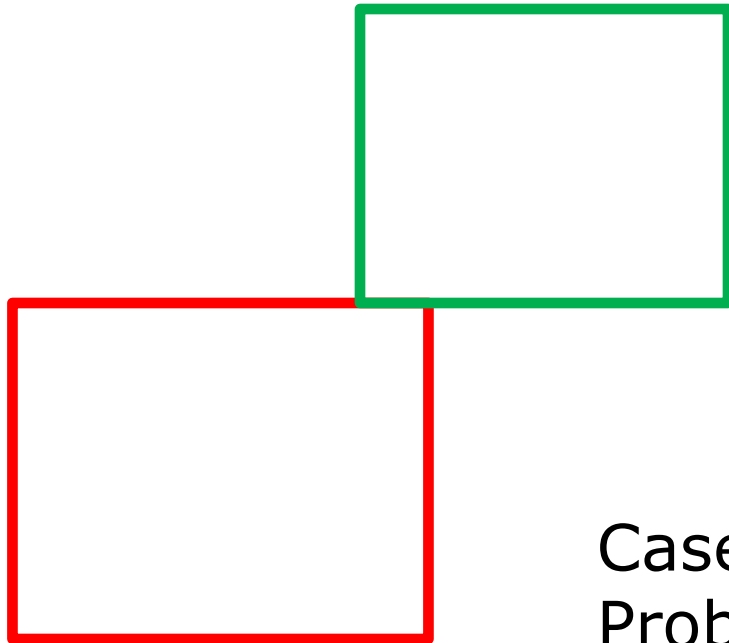
Case 1: Add Overlap
Problem: Bad Physics

Tricky Resolution Cases



Collision Detected
What to do? Have Options

Tricky Resolution Cases



Case 2: Move Up Only
Problem: Came from right

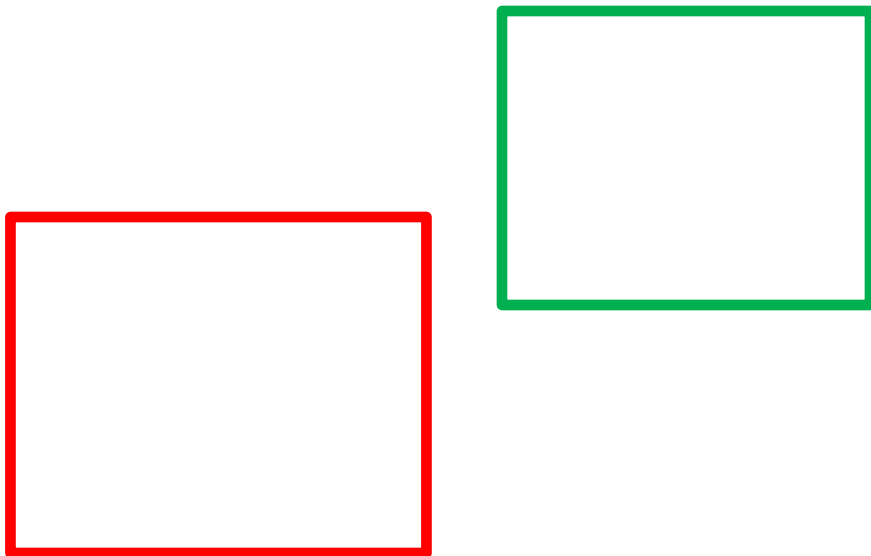
Tricky Resolution Cases



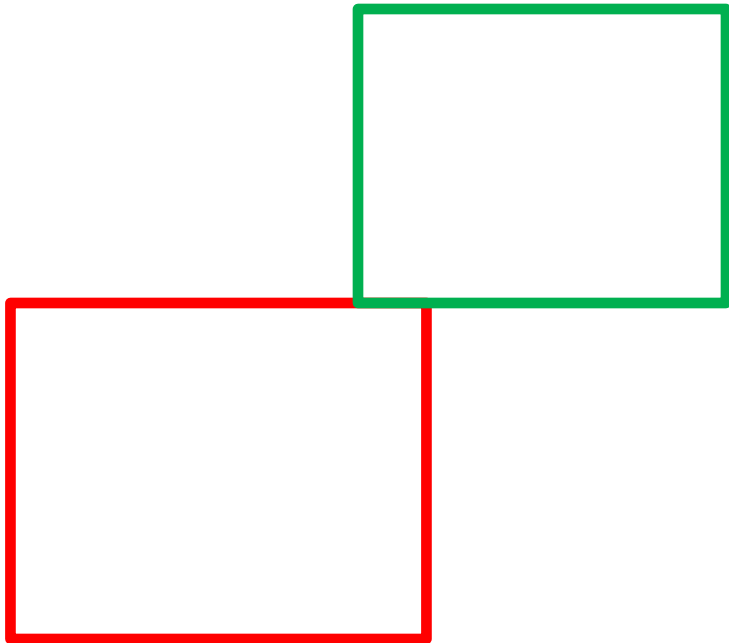
Tricky Resolution Cases



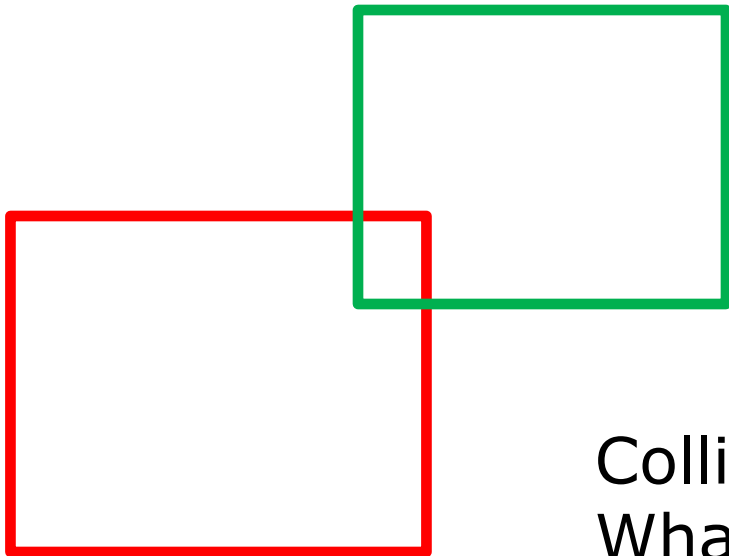
Tricky Resolution Cases



Tricky Resolution Cases

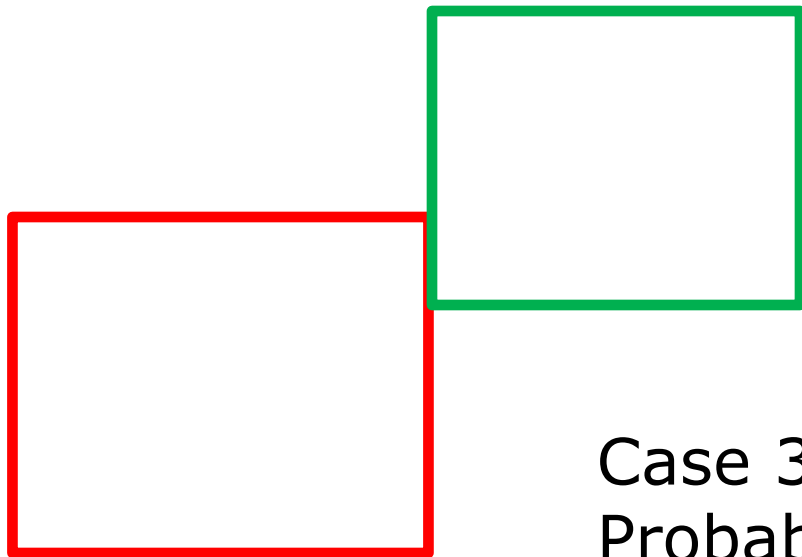


Tricky Resolution Cases



Collision Detected
What to do? Have Options

Tricky Resolution Cases



Case 3: Move Right Only
Probably Best 'Physics'

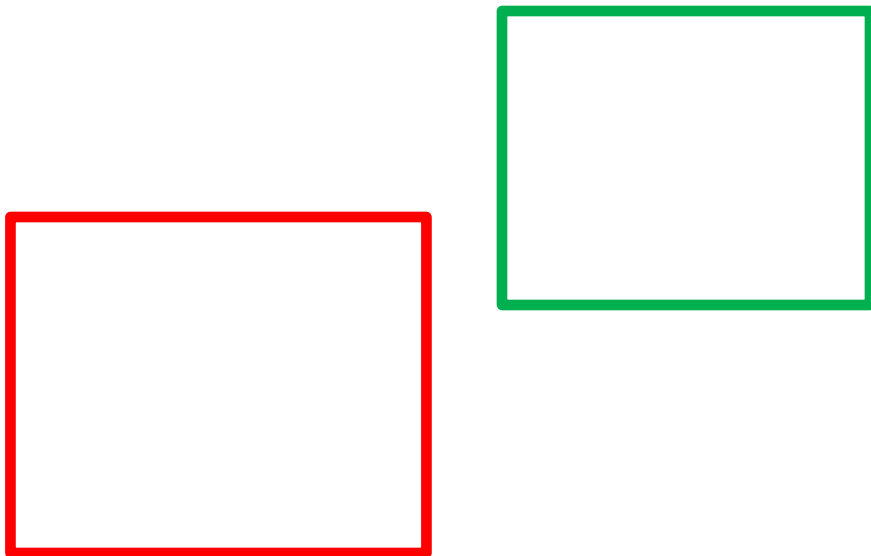
Tricky Resolution Cases



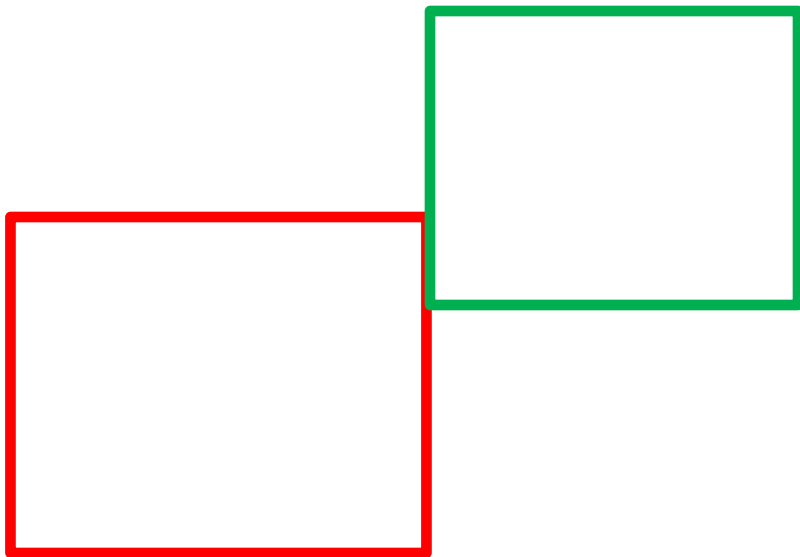
Tricky Resolution Cases



Tricky Resolution Cases



Tricky Resolution Cases

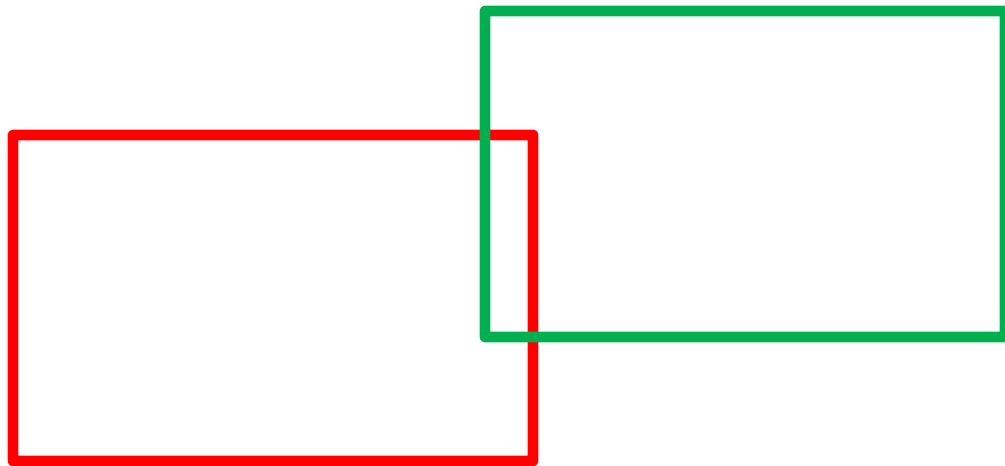


Detecting Collision Direction

- When detecting collisions in our game, they will first happen on a specific **frame**
- The frame **before** the collision is detected, there will be no collision
- Could try to use overlap to detect which direction the movement came from?

Tricky Resolution Cases

Use overlap to detect direction of movement?



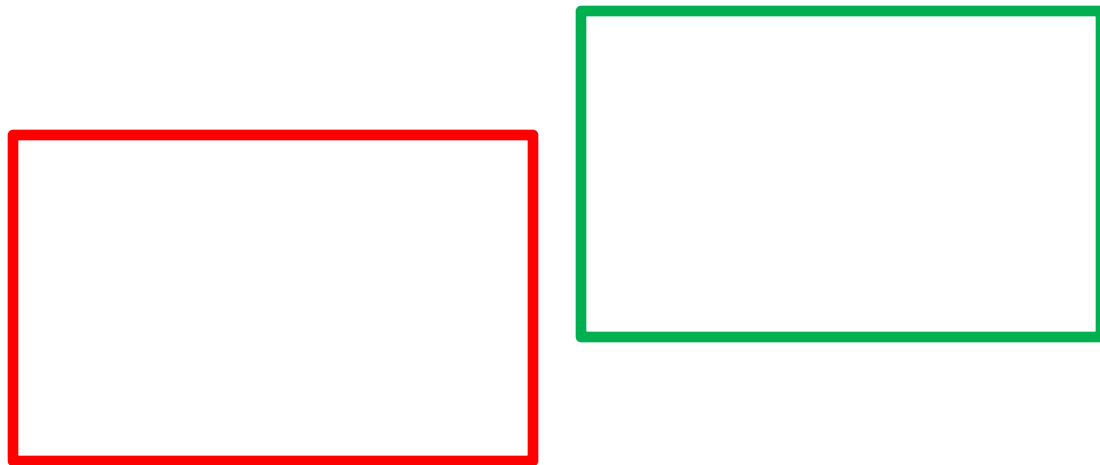
Tricky Resolution Cases



Problem: Could have come fast from y direction

Tricky Resolution Cases

Problem: Could have come
slow from x direction



Detecting Collision Direction

- When detecting collisions in our game, they will first happen on a specific frame
- The frame before the collision is detected, there will be no collision
- ~~• Could try to use overlap to detect which direction the movement came from?~~
- Use previous overlap to detect direction

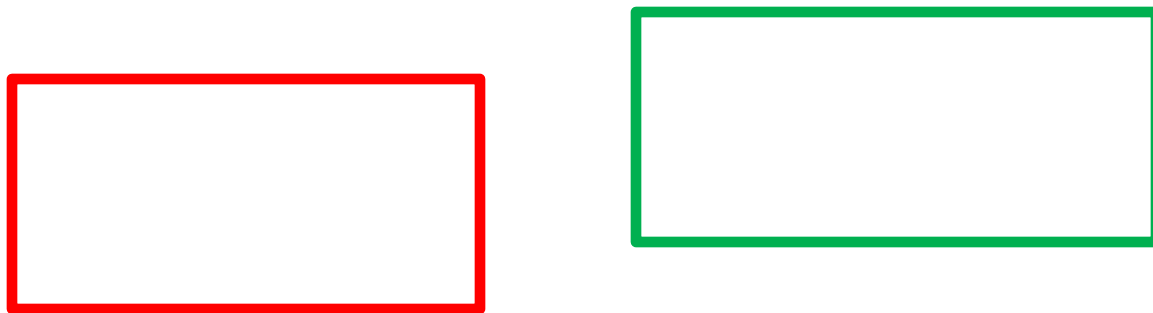
Using Previous Overlap

- If previous frame $\text{overlap.y} > 0$, then movement came from the side
- Resolve collision by pushing in x direction



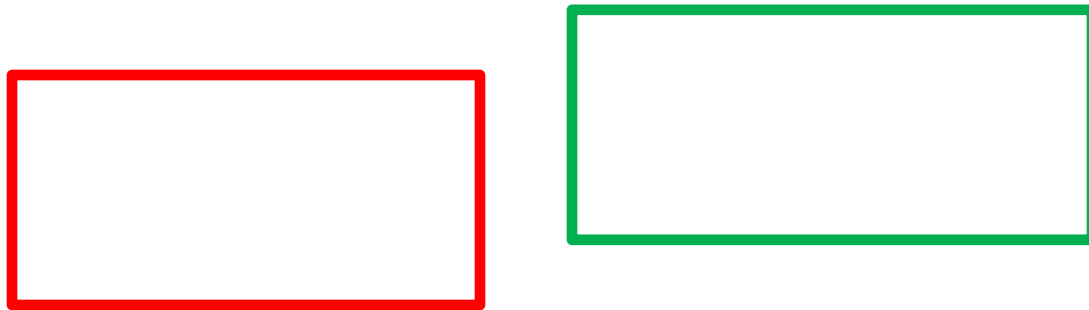
Using Previous Overlap

- If previous frame $\text{overlap.y} > 0$, then movement came from the side
- Resolve collision by pushing in x direction



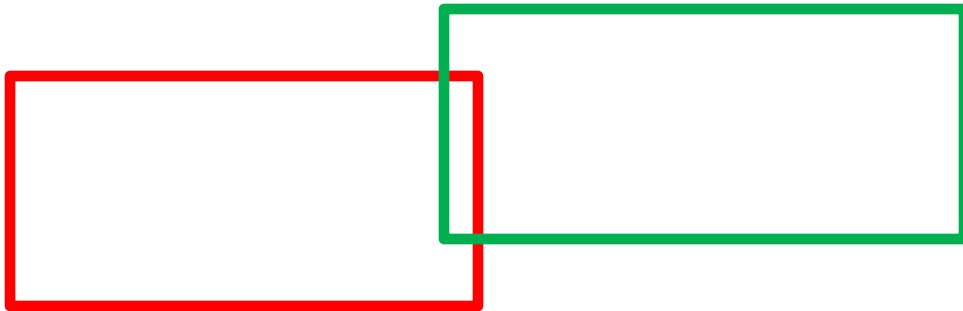
Using Previous Overlap

- If previous frame $\text{overlap.y} > 0$, then movement came from the side
- Resolve collision by pushing in x direction



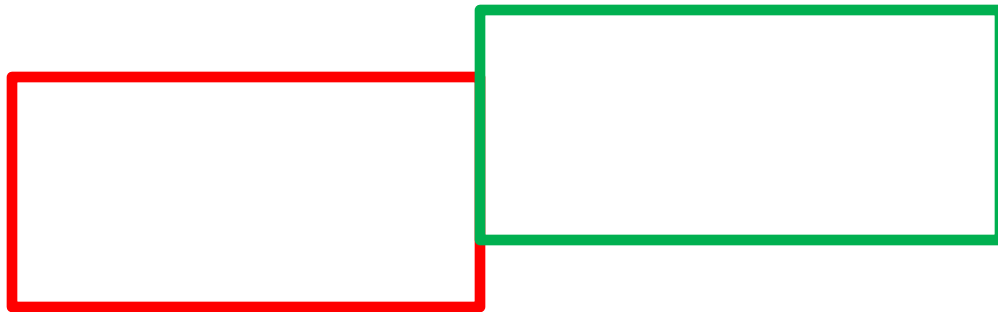
Using Previous Overlap

- If previous frame $\text{overlap.y} > 0$, then movement came from the side
- Resolve collision by pushing in x direction



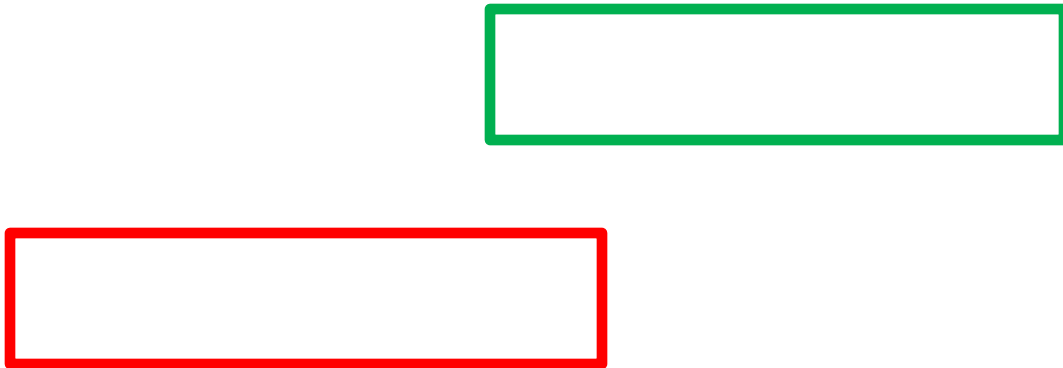
Using Previous Overlap

- If previous frame $\text{overlap.y} > 0$, then movement came from the side
- Resolve collision by pushing in x direction



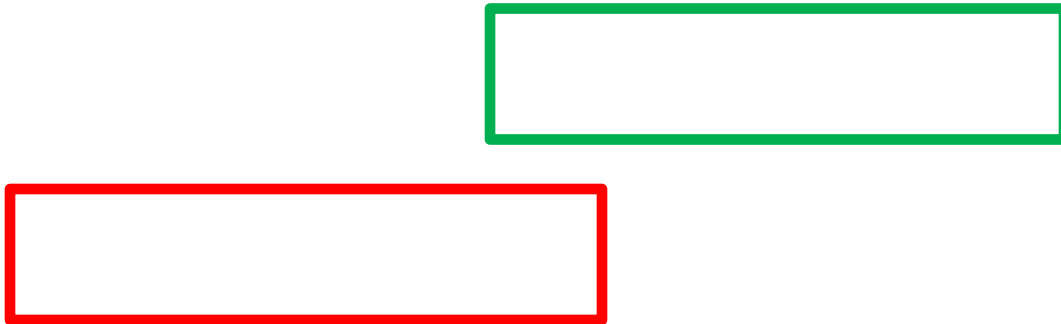
Using Previous Overlap

- If previous frame $\text{overlap.x} > 0$, then movement came from the top or bottom
- Resolve collision by pushing in y direction



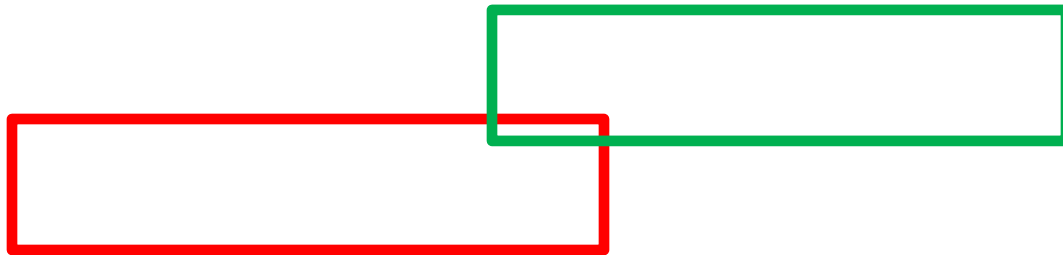
Using Previous Overlap

- If previous frame $\text{overlap.x} > 0$, then movement came from the top or bottom
- Resolve collision by pushing in y direction



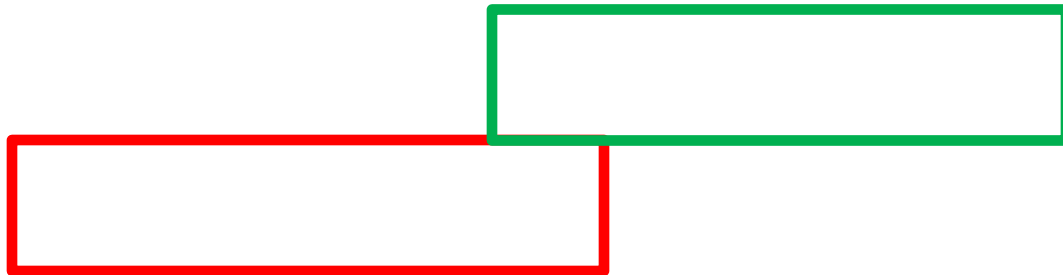
Using Previous Overlap

- If previous frame $\text{overlap.x} > 0$, then movement came from the top or bottom
- Resolve collision by pushing in y direction



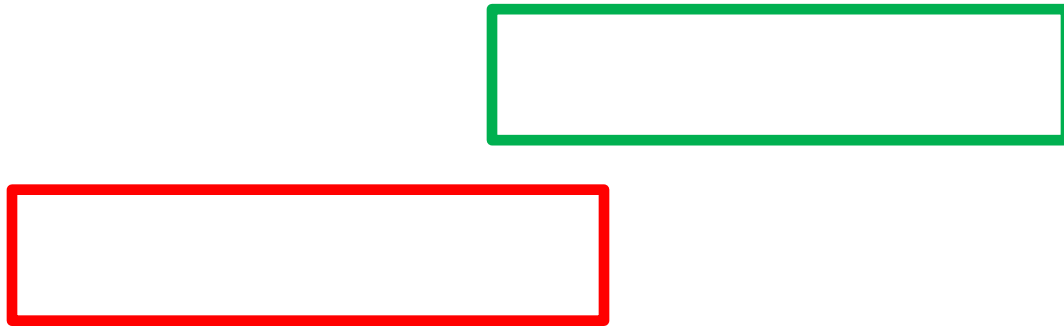
Using Previous Overlap

- If previous frame $\text{overlap.x} > 0$, then movement came from the top or bottom
- Resolve collision by pushing in y direction



Top / Bottom Detection

- A collision came from the top if it had a previous overlap $x > 0$ and its y is higher



Top / Bottom Detection

- A collision came from bottom if it had a previous overlap $x > 0$ and its y is lower



Tricky Resolution Case

- What if the previous frame had no overlap in either x or y direction?



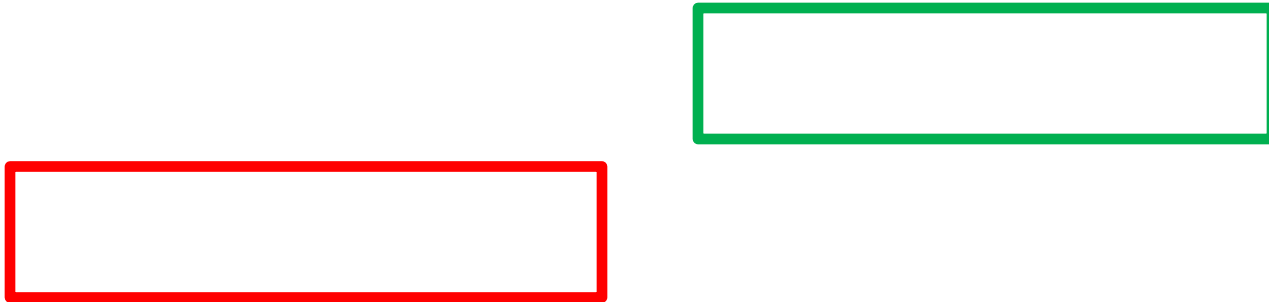
Tricky Resolution Case

- What if the previous frame had no overlap in either x or y direction?



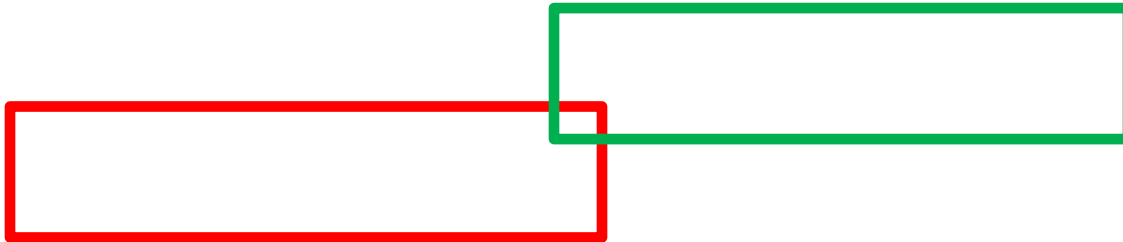
Tricky Resolution Case

- What if the previous frame had no overlap in either x or y direction?



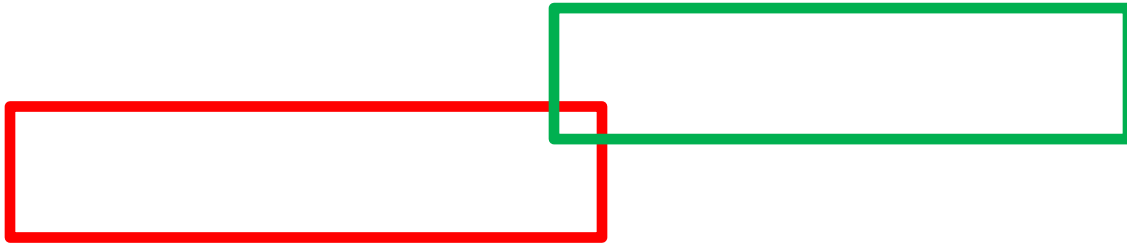
Tricky Resolution Case

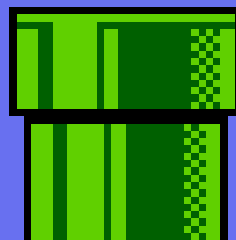
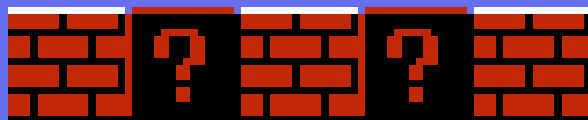
- What if the previous frame had no overlap in either x or y direction?



Tricky Resolution Case

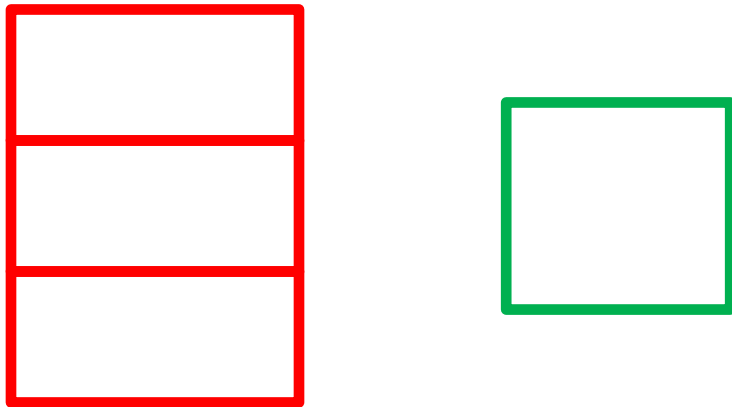
- What if the previous frame had no overlap in either x or y direction?
- Must decide how to resolve collision based on previous position and/or overlap size





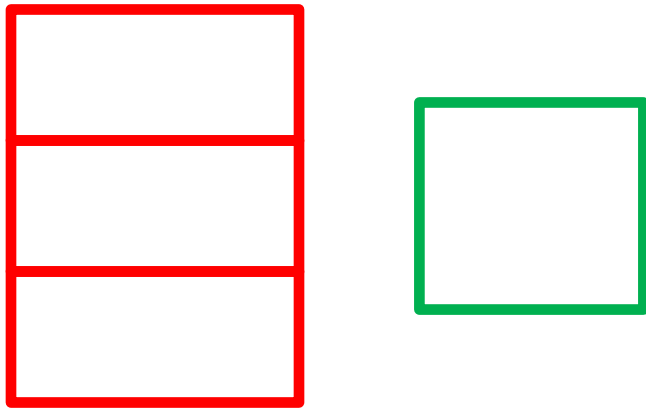
Tricky Resolution Case

- What if an entity overlaps with multiple entities? Ex: player moving near stack



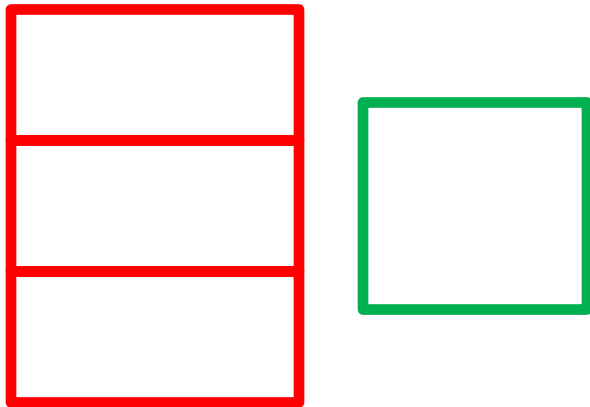
Tricky Resolution Case

- What if an entity overlaps with multiple entities? Ex: player moving near stack



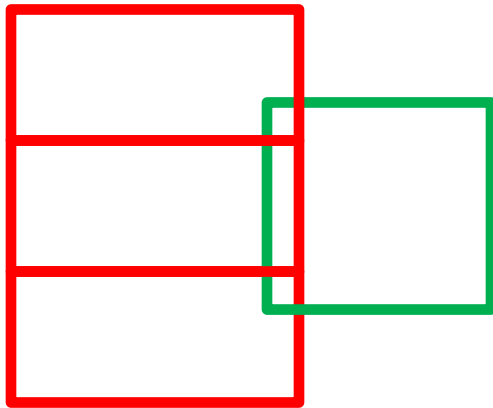
Tricky Resolution Case

- What if an entity overlaps with multiple entities? Ex: player moving near stack



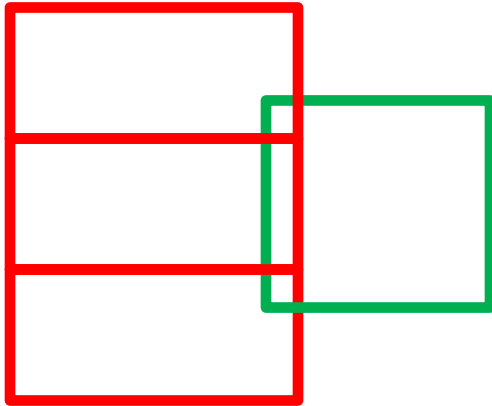
Tricky Resolution Case

- What if an entity overlaps with multiple entities? Ex: player moving near stack



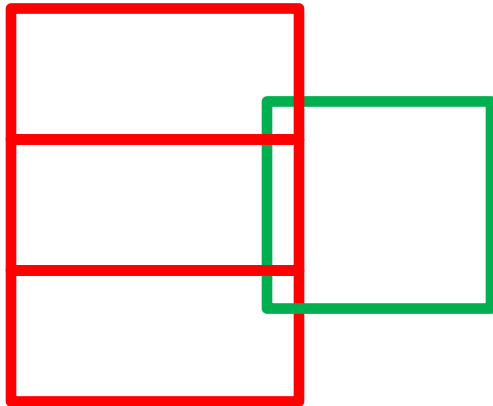
Tricky Resolution Case

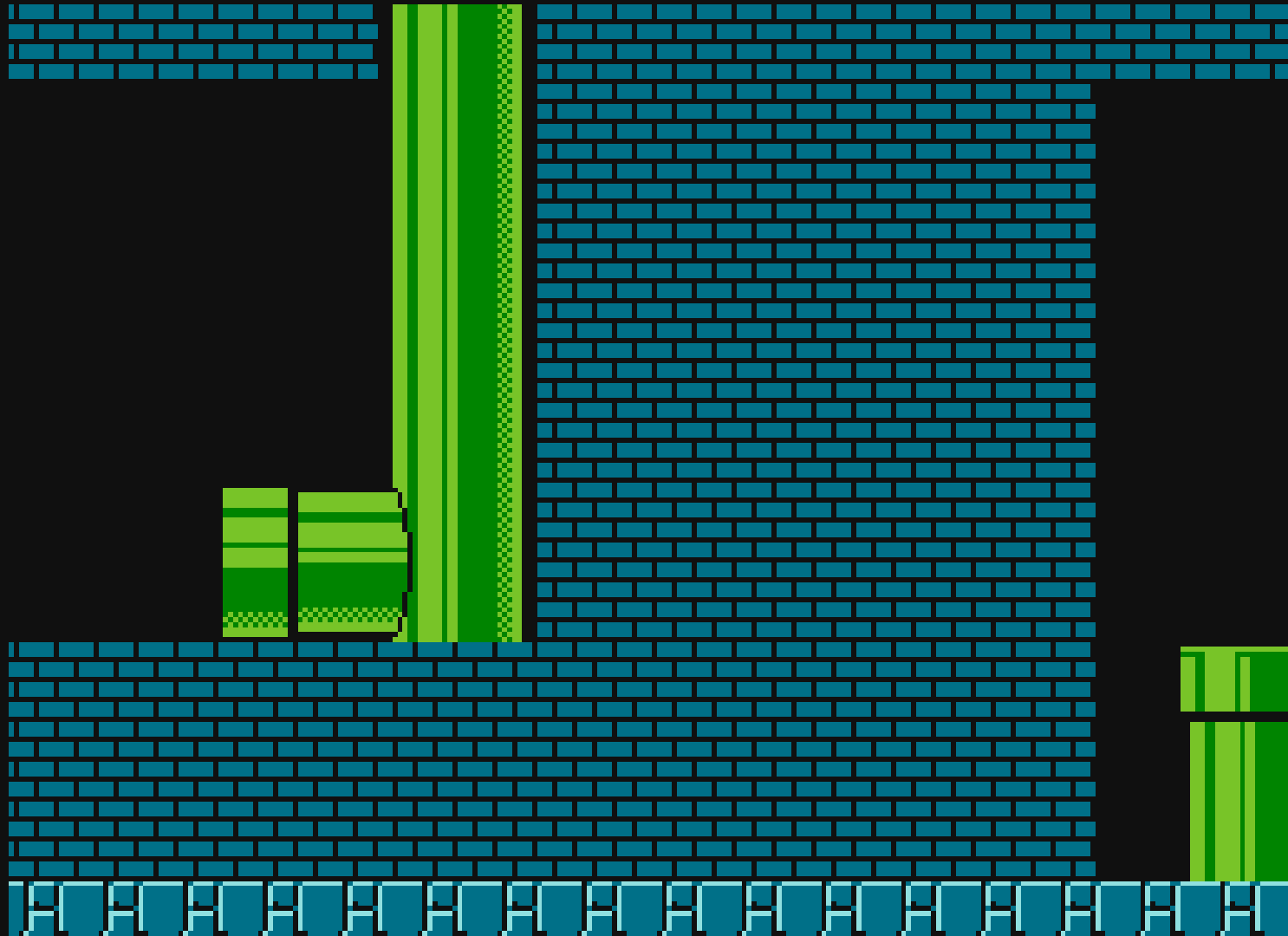
- Order of collision checks may decide which resolution occurs, can be tricky



Tricky Resolution Case

- The 'previous overlap' method discussed here usually resolves this case, using previous position only does not





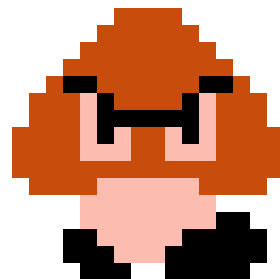
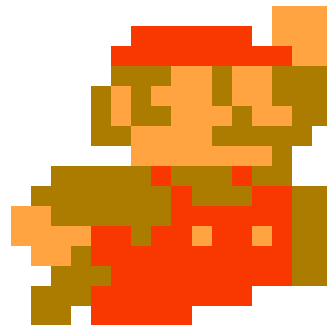
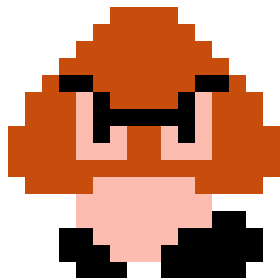
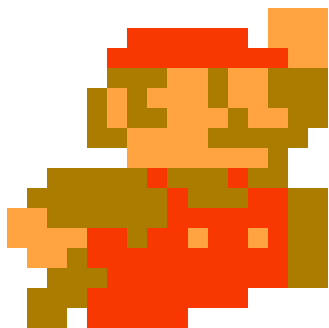
Important Note

- All collision resolution discussed here is for the simple case of avoiding overlap
- Useful for: walking on tiles, not being able to pass through tiles, bullet hits entity
- Many other kinds of collision resolution:
 - Kinematic object elastic / inelastic collision
 - Physics simulators with mass / forces

Side Note: Super Mario Bros

- Super Mario Bros collisions operate slightly differently than we presented
- Consider the case of Mario jumping on top of an enemy, how is that detected?
- Same AABB overlap is used, however Mario kills the enemy if he is moving downward (with x-direction ignored)

Super Mario Bros Collision



Recommended Videos

- “How is this speedrun possible?”
YouTube: Bismuth
https://youtu.be/FQJEzJ_cQw
- “Walls, Floors, & Ceilings”
YouTube: Pannenkoek
<https://youtu.be/UnU7DJXiMAQ>
- SM64 - Watch for Rolling Rocks
YouTube: Pannenkoek
<https://youtu.be/kpk2tdsPh0A>